Calibrate the Level, Borrow Across Pairs, Learn Items Online: A Random-Effects Predictor for the Predictive AI Evaluation Competition

Neel Sirivara
University of Washington
nsiriv@cs.washington.edu

Abstract

We describe our selected submission to the Predictive AI Evaluation Competition (PAIEC), Codabench submission 1019682. From the evaluator's *global* list of acquired labels, it predicts whether an AI system answers a benchmark item correctly. It has three layers: (i) random effects for benchmark, subject, subject–benchmark pair and exact item, pooled across all sampled pairs; (ii) a budget-0 level, recalibrated after formative feedback showed our earlier prior was worse than predicting 0.5; (iii) an item layer learned online within each benchmark. In pre-registered, benchmark-held-out simulations of the official protocol, the item layer lowers the Brier area under the learning curve (ALC) by -0.0035 [-0.0051, -0.0022] on inventory benchmarks. On the hidden pool, the base and selected predictors average 0.1904 and 0.1705 ALC. That gap is mostly draw composition: it is already -0.0208 at budget 0, where the two predictors are identical. A post-hoc comparison over shared pairs gives -0.0024 (se 0.0016). That matches the simulated size but is not significant. An LLM difficulty rating and a sentence-embedding kernel failed their pre-registered gates. Development used Claude Code as a coding and research assistant (Section 9).

1 Introduction

Evaluating every new AI system on every benchmark is expensive, and the number of benchmarks keeps growing [Liang et al., 2023]. Recent work therefore predicts performance instead of measuring it. The prediction can come from a few informative items selected with item response theory (IRT) [Polo et al., 2024, Kipnis et al., 2025, Perlitz et al., 2024], from items chosen adaptively for each model [Hofmann et al., 2025, Zhuang et al., 2025], from other models' scores [Ruan et al., 2024, Ye et al., 2023], or from a few labelled instances of the new model [Pacchiardi et al., 2024]. PAIEC [Stanford AIMS Lab, 2026] turns this into a forecasting task with a fixed interface. `predict([subject, item], labeled)` returns $\Pr(\text{correct})$ for an item of a benchmark in the private test pool. `labeled` holds the labels acquired so far, at budgets of $n \in \{0, 1, 3, 7, 15, 31\}$ labels per subject–benchmark pair.

Contributions.

- **A three-layer predictor for the global label list** (Section 3, Figure 1): cross-pair random effects, a recalibrated budget-0 level and an online, within-benchmark item layer. We selected the layers in a benchmark-held-out simulator of the official protocol. The item layer passed a pre-registered gate (Section 5.1). The level change followed formative feedback and was not pre-registered (Section 4).

Technical report, Predictive AI Evaluation Competition (PAIEC), 40th Conference on Neural Information Processing Systems (NeurIPS 2026).

- **A reading of noisy formative draws** (Section 5.2). Across 7 + 4 organiser-scored draws, the arm means are dominated by which pairs were drawn. A post-hoc pair-matched comparison recovers the simulated effect size. It does not recover the simulated significance or budget profile.
- **Negative results** (Section 5.3). An LLM's rating of item difficulty does not improve the predictor. Neither does a semantic text kernel, nor an exact Bayes pair-rate posterior.
- **A generated, audited report** (Section 8). A script generates every number from the released result files. A second script re-derives each directional claim from those files.

Related work. Item response models place subject ability and item difficulty on one logit scale. They are standard for LLM benchmark data [Rodriguez et al., 2021, Polo et al., 2024, Kipnis et al., 2025], but fit item parameters offline on hundreds of models. In PAIEC the test benchmarks are new, so difficulty must be learned *during* evaluation, from a few labels of a few subjects. We keep the Rasch structure [Lord, 1980] but replace its free parameters with Gaussian random effects, which gives a generalised linear mixed model [Breslow and Clayton, 1993]. Following work that predicts scores from other models' scores [Ruan et al., 2024, Ye et al., 2023], we also pool across subjects within one evaluation run.

2 Task and evaluation protocol

Task and metric. A *subject* is a dictionary of strings (`normalized_name`, `provider`, `release_date`, `access_date`, `harness`, `harness_version`, `reasoning_effort`, `subject_features_extra`). An *item* has `item_content`, `item_features`, `interactors` and an anonymous `benchmark_id`. Each subject–benchmark pair with at least 80 distinct items is split 50/50 into an acquisition pool and an evaluation pool, fixed across submissions. Acquisition candidates arrive in a fixed random order, and an optional `acquisition_function` decides which labels to reveal, at most 31 per pair. After 0, 1, 3, 7, 15 and 31 labels per pair, every evaluation item is predicted. This gives Brier scores [Brier, 1950] $B_0, \ldots, B_{31}$ and

$$\mathrm{ALC} = 0.1\,B_0 + 0.2\,B_1 + 0.2\,B_3 + 0.2\,B_7 + 0.2\,B_{15} + 0.1\,B_{31}, \tag{1}$$

averaged over pairs; lower is better. Predicting 0.5 gives $B_n = 0.25$ on any data.

The global label list. In the organisers' streaming client (`run_streaming`), every prediction at a checkpoint receives the *same* `labeled` list. In the competition page's words, it "contains [[subject, item], label] entries across sampled subject–benchmark pairs". At budget n it holds up to n labels of *every* pair; at budget 0 it is empty. The rules permit this evidence: "the submitted program may use the attributes and acquired labels supplied through the evaluation interface to adapt during evaluation". The model uses only anonymous keys within one run and keeps nothing across runs.

Data.

- **Training release:** `aims-foundations/measurement-db`, revision `54074453da`. It holds 16,718 pairs of (subject, test condition) on 144 benchmarks with at least 20 items, with 3,118,742 item-level labels on 165,764 distinct items. We use it only to simulate the protocol and select constants; no regression coefficient of the submission is fitted to it. Its 33 benchmarks that are rows of the organisers' benchmark inventory are harder than the rest (mean pair success rate 0.466 vs. 0.547). Competition-eligible base models are 2025–2026 releases [Stanford AIMS Lab, 2026].
- **Hidden test pool:** each of our formative runs scored 8–9 pairs, with 487–498 evaluation subject–items, on the same 7 benchmark IDs. Pairs recur across runs and keep their evaluation items (Appendix A).
- **Local testbed:** 17 pairs of four small open models on GSM8K, SVAMP and BIG-Bench-Hard. They serve as an engineering check inside the organisers' client (Appendix F).

Simulator. We evaluate the cross-pair model, the level and the item layer in a simulator of the official protocol built on the training release (Appendix D).

- The release's benchmarks are partitioned into folds, and every result is benchmark-held-out.
- A *test set* is a group of pairs from one fold that share one global list.
- Each pair's items are split 50/50, and labels are acquired in random order, the evaluator's default. Our fresh-first rule (Section 3) would change the ALC by -0.0005 [-0.0009, -0.0002], and the simulations below do not use it.

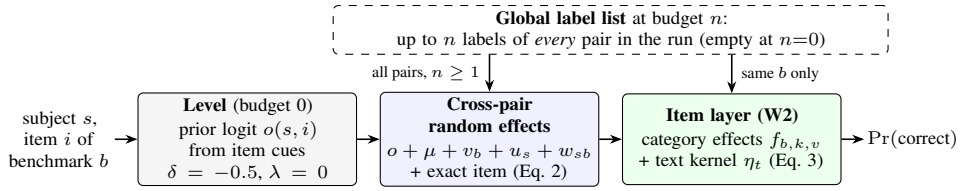


Figure 1: Structure of the submitted predictor. At budget 0 only the level is used. From budget 1 on, the cross-pair model pools all pairs of the run, and the item layer adds within-benchmark item effects. Each layer shrinks towards the one on its left. Any failure falls back to the per-pair predictor, and then to 0.5.

- At budget n, every prediction receives the first n labels of every pair in the test set.

A *population* restricts the benchmarks or the shape of the test sets:

- **A**: all benchmarks, formative-like test sets.
- **INV**: the 33 inventory benchmarks, our proxy for the hidden pool.
- **MID**, **HARD**: the harder benchmarks of A.
- **Y25**, **INV25**: A and INV, recent subjects only.
- **SHIFT**: A with an over-optimistic prior, in which every prior logit is raised.
- **SPARSE**, **H7**, **H7-INV**: test sets shaped like our formative runs.

The populations share benchmarks: they are overlapping views of the same folds, not independent replications.

Statistics and pre-registration. The unit of analysis is a (test set, pair). Two predictors are compared on identical labels by the mean ALC difference over units, with a 95% percentile bootstrap CI. Units from one benchmark are correlated, so the bootstrap resamples whole benchmarks (2,000 resamples). *Pre-registration* means a dated file in our repository, written before the runs it governs, whose SHA-256 the result files record. None was deposited with a third party.

3 The submitted predictor

The predictor stacks three layers (Figure 1), and each layer shrinks to the one below it when evidence is thin. Table 5 (Appendix A) lists the level constants of all scored variants.

Budget-0 level. The prior logit is $o(s,i) = \mathrm{clip}\big(\lambda\, z_{\mathrm{subj}}(s) + \delta + z_{\mathrm{item}}(i), \pm 1.5\big)$, and at budget 0 the prediction is $\sigma(o)$.

- z_{item} reads the tier, level and points cues in `item_features`, and the item length.
- The hand-set z_{subj} rewards a recent release, high reasoning effort, a large model and a non-default harness (Appendix E).

The submission sets $\lambda = 0$ and $\delta = -0.5$. Every subject therefore starts from the same level, and an item without cues receives $\sigma(-0.5) = 0.38$.

Cross-pair random-effects model. From budget 1 on, a label y_e in the global list, of subject s on item i of benchmark b, is modelled as

$$\mathrm{logit}\ \Pr(y_e = 1) = o(s,i) + \mu + v_b + u_s + w_{sb}, \tag{2}$$

with independent Gaussian priors of standard deviations $\tau_\mu = 2$, $\tau_b = 1.5$, $\tau_s = 0.5$ and $\tau_{sb} = 0.6$. A subject is identified by its complete attribute dictionary, and a benchmark by its `benchmark_id`.

- **Fitting.** The MAP estimate is computed by backfitting, with one Newton step per effect type per sweep and at most 25 sweeps (penalised quasi-likelihood [Breslow and Clayton, 1993]).
- **Prediction.** The target's latent logit m and approximate posterior variance V give $p = \sigma\big(m/\sqrt{1 + \pi V/8}\big)$ [MacKay, 1992].
- **Exact-item effect.** The same item (same `benchmark_id` and text) may already be labelled for other subjects. Then an item effect with prior sd $\tau_d = 2.25$ is estimated from those labels by 4 Newton steps, and added on the conditional scale (Appendix D).

Item layer (W2). The layer has two parts. Both are fitted online on the current list, within each benchmark.

- **Category effects.** An `item_features` key is *active* for a benchmark if at least two of its values occur among that benchmark's labelled items. For each active key, a random effect $f_{b,k,v} \sim \mathcal{N}(0, 0.5^2)$ is added to Eq. 2 and fitted jointly. A target adds the posterior mean and variance of its own (benchmark, key, value) group; an unseen value adds nothing. The grid's cap on values per item (1) does not bind.
- **Text kernel.** For a target t on benchmark b, the kernel pools the stage-1 residuals of the other labelled items j of b. Stage 1 is Eq. 2 with the category effects, before the exact-item effect:

$$\eta_t = \frac{\sum_j K_j G_j}{\sum_j K_j H_j + 1/\tau_k^2}, \qquad K_j = \cos(t, j)^q. \tag{3}$$

  Here G_j is item j's summed residual $y - p$, and H_j its summed weight $p(1-p)$. cos is the cosine of hashed character-4-gram TF-IDF vectors, with idf from b's labelled items. The constants are $\tau_k = 1$ and $q = 2$. The prediction becomes $\sigma(\mathrm{logit}\, p + \eta_t)$.

At budget 0 the list is empty, so the prediction is unchanged.

Acquisition and implementation.

- **Acquisition.** `labeling.py` accepts candidates in stream order, a uniformly random sample. While budget remains, it skips any item already labelled for another subject. It takes no `prediction` argument, so the client never calls `predict` during acquisition.
- **Dependencies and speed.** `model.py` and `labeling.py` are deterministic and use only the Python standard library, with no data files, models or network. The fit is cached per checkpoint. On a 62,000-label list, the first call takes 6.5 s and later calls 4.9 ms on one laptop core.
- **Fallbacks.** On any failure, the predictor falls back to the base prediction, then to our earlier per-pair predictor (Appendix E), then to 0.5.

4 Design decisions

Recalibrating the level. The organisers' feedback on our round-4 submission 1010370 gave $B_0 = 0.2834$, *above* the constant's 0.25 on the same draw. Its budget-0 ECE (0.340) also exceeded the 0.235–0.299 of four simulated populations. The training release shows the same failure. On its 33 inventory benchmarks, the hand-set prior is worse than the constant at budget 0 ($B_0 \approx 0.2596$), and its release-year term raises every eligible test subject.

We therefore compared variants in seven simulated populations, from release-like to harder and inventory-like (Table 1). The selected variant 1010717 removes the subject terms ($\lambda = 0$) and sets $\delta = -0.5$. Its ALC change is:

- -0.0069 [-0.0090, -0.0048] on HARD, and -0.0041 on MID;
- not significantly different from zero on INV;
- +0.0018 [+0.0005, +0.0032], a cost, if the pool is release-like.

The level matters almost only at budget 0; later budgets change by at most 0.0046. Of the three variants we uploaded, 1010717 gains more than 1010715 on INV, MID and HARD, and costs less than 1010735 on A, Y25 and INV. The formative draws could not separate them (Appendix A). We made this choice after the formative feedback, and it was not pre-registered. Appendices A and C give the details and the other variants.

Why an item layer. Two analyses locate the remaining error.

(a) The pair-rate component leaves little to gain. In exploratory runs shaped like the hidden runs (configuration H7), an exact Bayes grid posterior changes the ALC by +0.0017 [-0.0024, +0.0058] on H7-INV. Its gain on H7 comes entirely from budget 0, that is, from a different level. Faster updating and other prior sds do not help either (Appendix B).

(b) Item difficulty holds most of the headroom. In simulation, oracle item effects would change the ALC by -0.0417, against -0.0065 for oracle benchmark levels (Appendix C). With unlimited labels (all release subjects; leave-one-item-out within each benchmark), the share of within-benchmark

Table 1: Simulated ALC change of the level variants against 1010370 (round-4 simulator, benchmark-held-out; populations as in Section 2); negative is better. Variants (Table 5): 1010715 turns the subject terms off; 1010717 also sets $\delta = -0.5$; 1010735 also sets $\delta = -1$. 1010717: mean [95% cluster-bootstrap CI over benchmarks]. Other variants: means, with CIs in Table 8. *: CI excludes 0, not adjusted for the number of cells. Rate: median pair success rate. B_0: budget-0 Brier, 1010370 $\rightarrow$ 1010717.

			ALC		ALC change from 1010370		
Population	Bench.	Rate	1010370	B_0	1010715	**1010717** [95% CI]	1010735
A (release-like)	135	0.52	0.1965	0.247→0.262	+0.0002	+0.0018* [+0.0005, +0.0032]	+0.0057*
Y25 (2025+ subjects)	55	0.62	0.1860	0.245→0.286	+0.0007	+0.0046* [+0.0015, +0.0076]	+0.0104*
INV (inventory)	33	0.36	0.1887	0.256→0.241	-0.0010	-0.0020 [-0.0050, +0.0015]	-0.0003
INV25 (INV, 2025+)	16	0.42	0.1898	0.260→0.256	-0.0015	-0.0009 [-0.0063, +0.0052]	+0.0019
MID (rate ≤ 0.5)	62	0.28	0.1903	0.251→0.215	-0.0011*	-0.0041* [-0.0057, -0.0025]	-0.0042*
HARD (rate ≤ 0.4)	41	0.20	0.1630	0.248→0.190	-0.0019*	-0.0069* [-0.0090, -0.0048]	-0.0087*
SHIFT (A, prior +1)	135	0.52	0.2023	0.297→0.258	-0.0006	-0.0042* [-0.0056, -0.0028]	-0.0055*

Table 2: Item layer (W2) against its base 1010717 on the confirmation folds (benchmark-held-out). Columns: ALC; the difference with a 95% cluster-bootstrap CI over benchmarks (2,000 resamples); per-budget Brier differences (B_0 is 0 by construction). Gate roles are as pre-registered. The scenarios share benchmarks (INV and SPARSE use the inventory benchmarks; MID and HARD are subsets of A), so the rows are not independent replications.

Scenario	Bench.	base	W2	ΔALC [95% CI]	ΔB_1	ΔB_3	ΔB_7	ΔB_{15}	ΔB_{31}	gate
INV	20	0.1922	0.1887	-0.0035 [-0.0051, -0.0022]	-0.0014	-0.0021	-0.0042	-0.0066	-0.0065	(a) CI < 0
A	83	0.1952	0.1929	-0.0024 [-0.0032, -0.0016]	-0.0011	-0.0016	-0.0029	-0.0040	-0.0043	(b) not worse
SPARSE	20	0.1999	0.1978	-0.0021 [-0.0035, -0.0009]	-0.0003	-0.0012	-0.0021	-0.0044	-0.0053	(c) not worse
MID	39	0.1855	0.1836	-0.0019 [-0.0029, -0.0009]	-0.0011	-0.0013	-0.0024	-0.0028	-0.0035	reported
HARD	26	0.1556	0.1536	-0.0019 [-0.0033, -0.0009]	-0.0009	-0.0008	-0.0025	-0.0034	-0.0040	reported

item-effect variance that a signal explains is 0.118 for `item_features` categories and 0.048 for the character-4-gram kernel. For cheap text statistics (length, digits, options) it is 0.019, so we dropped them.

5 Results

5.1 Simulator: pre-registered tuning and confirmation of the item layer

Protocol. `experiments/results/w2_items/PREREG.md` (written 2026-10-29 10:58 PDT, SHA-256 prefix `192748b713c4`) fixed five things before the tuning grid and any confirmation run: the layer, the scenarios, the grids, the selection criterion and the gate. Benchmark folds 0–1 tune and folds 2–4 confirm; the folds partition the benchmarks.

Tuning. On the tune folds, 18 of the 18 settings lowered the mean ALC difference over A, INV, MID and HARD. The combination of the best category setting and the best kernel setting was the best (-0.0016; categories alone -0.0011, kernel alone -0.0008).

Gate and confirmation. The gate passes on the confirmation folds (Table 2). Each pre-registered condition, with its result:

(a) *INV: 95% cluster-bootstrap CI below 0.* Result: -0.0035 [-0.0051, -0.0022].
(b) *A: no significant loss.* Result: -0.0024 [-0.0032, -0.0016].
(c) *SPARSE, nine pairs on seven inventory benchmarks shaped like a formative run: no significant loss.* Result: -0.0021 [-0.0035, -0.0009].

Budget 0 is unchanged in every scenario, and the gain grows with the budget, as more labelled neighbours become available. On INV, the confirmation gain is larger than the tuning gain (-0.0010). We attribute this to benchmark-to-benchmark variation and do not rely on it.

Table 3: Organiser-scored formative draws (Codabench Scoring output, means over pairs). Rows: the mean over draws per arm (sd over draws); the difference of the arm means (Welch se); the post-hoc pair-matched difference over subject–benchmark pairs present in both arms (se over pairs, treating pairs as independent); the simulator's expectation for formative-shaped test sets (SPARSE). Table 4 lists every draw.

	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC
base (`xp_level_m050`), 7 draws	0.2415	0.2052	0.1879	0.1822	0.1726	0.1665	0.1904 (sd 0.0187)
selected (W2), 4 draws	0.2207	0.1886	0.1703	0.1571	0.1523	0.1481	0.1705 (sd 0.0136)
difference of arm means	-0.0208	-0.0166	-0.0176	-0.0251	-0.0203	-0.0184	-0.0199 (se 0.0098)
pair-matched difference (11 pairs, post hoc)	+0.0000	-0.0114	+0.0013	+0.0009	-0.0012	-0.0035	-0.0024 (se 0.0016)
simulator, SPARSE (expected difference)	+0.0000	-0.0003	-0.0012	-0.0021	-0.0044	-0.0053	-0.0021

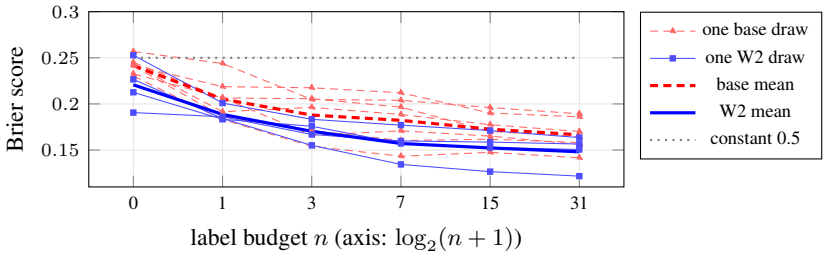


Figure 2: Formative learning curves. Thin lines are single draws (dashed red triangles: base 1010717; solid blue squares: selected W2), and thick lines are the arm means. The spread within an arm is draw noise. The two thick curves already differ at budget 0, where the two predictors are identical.

5.2 Formative draws on the hidden pool

Arm means. Each formative run scores a fresh draw of pairs, and the draw dominates the score (Table 3, Figure 2).

- **Base 1010717:** 7 draws, two of them deployment probes that return its predictions (Section 5.3). Mean 0.1904, standard deviation 0.0187, range 0.1628–0.2143.
- **Selected submission:** 4 counted draws (1019688 had not finished by the rule's contingency time). Mean 0.1705, standard deviation 0.0136, range 0.1549–0.1881.[1]

The arm means differ by -0.0199 (se 0.0098). Most of that is draw composition, not the item layer. The difference is already -0.0208 at budget 0, where the two predictors are identical, and it stays roughly flat across budgets. The simulator instead predicts a difference that is zero at budget 0 and grows with the budget.

Pair-matched comparison (post hoc). Runs share subject–benchmark pairs, and a pair keeps its evaluation items across runs, so matching on pairs removes most of the composition effect. This analysis was not part of the switch rule below. Over the 11 pairs that occur in both arms, the budget-0 difference is exactly 0. The ALC difference is -0.0024 (se 0.0016, $t = -1.50$), with 8 of 11 pairs better. The size matches the simulator (SPARSE -0.0021), but the budget profile does not:

- about 94% of the difference comes from B_1 (-0.0114, se 0.0028), where the simulated difference is at most 0.0014 in magnitude in every scenario;
- at B_3–B_{15}, where the simulator places most of the gain, the matched differences are within one standard error of zero (Table 3).

The se treats pairs as independent, although pairs of one run share a label list, and the matched pairs still differ in which other pairs shared their list. The real draws therefore neither confirm nor contradict the simulated effect.

[1] W2 run 1019686 was scored (its log is complete, 8 pairs) but ended with the platform's infrastructure archival error (`PAIEC-INFRA-ARCHIVE-001`), so the platform recorded no score. Under a retry note added to `SWITCH_RULE.md` after this failure (with its complete log at hand) and before the retry's result was retrieved, it was retried once with the same ZIP as 1019903. Its log-only ALC (0.1581) is not counted; this cannot favour the switch, because it is below the W2 arm mean (0.1705) (Appendix A).

Selection. A rule written before any W2 draw returned (`SWITCH_RULE.md`) switches the submission to the first W2 run that finishes, if three conditions hold:

(1) all 5 W2 runs finish without a platform error;
(2) the W2 mean ALC is worse than the base mean by at most 0.022, two standard errors of the difference of arm means (computed with the base arm's standard deviation);
(3) no per-budget mean is worse by more than 0.03.

A contingency clause, added while no W2 draw had finished, applies conditions 2 and 3 to the finished runs if some are unfinished at a fixed time. Condition 2 holds (-0.0199), condition 3 holds (largest per-budget difference -0.0166), and 1019688 had not finished by the rule's contingency time (Oct 30 12:00 PDT), so the contingency replaces condition 1. Under the contingency, conditions 2 (tolerance recomputed for 4 runs: 0.023) and 3 are applied to the 4 finished runs and both hold, so we select 1019682, the first W2 run that finished, as the rule specifies. Its own draw scored 0.1881, the highest (worst) ALC of its arm (mean 0.1705), so the selected ID is not a best-of-draws pick. The draws screen only for deployment failures and gross regressions; the selection rests on the pre-registered simulator gate.

Leaderboard context. The public board (snapshot 2026-10-26) has 58 entries, each a user's *best* draw: #1 0.1172, #10 0.1483, median 0.1768. Our arm means would rank about 28 (W2) and 46 (base). Both are far behind the leaders, a gap the item layer cannot close.

5.3 Negative results

- **LLM-rated item difficulty** (pre-registered 2026-10-26 13:08 PDT; three amendments before any rating was analysed: 2026-10-26 13:18 PDT, 2026-10-26 13:31 PDT, 2026-10-26 14:08 PDT). *Setup:* Qwen/Qwen3-VL-4B-Instruct rated all 24,199 items of INV and of a release-like check set, AS, in one forward pass each, scored as $s_1 = \log P(\text{Yes}) - \log P(\text{No})$ for "would a state-of-the-art 2025–26 AI system most likely complete this task correctly?". *Result:* a leave-benchmark-out offset $w\, z(s_1) + b$ on top of 1010717 fails its gate. Against the base it gives INV +0.0005 [-0.0015, +0.0024] and AS -0.0023 [-0.0050, +0.0008]; against an intercept-only control, INV -0.0006 [-0.0014, +0.0002]. *Diagnosis:* the slope is positive in 5 of 5 folds, but the within-benchmark correlation with the empirical item effect is only 0.089 (INV) and 0.037 (AS) (Appendix C). After this gate failed, amendments registered Gemini and Groq API raters, which were never run, and a local rater. A larger local rater, Qwen3-14B (4-bit), was pre-registered (Amendment 6, 2026-10-29 12:53 PDT) but not evaluated: rating stopped at 4,865 of 20,655 prompts because of contention on the shared machine before the deadline, so its gate never ran; the partial ratings are kept.
- **Semantic kernel (W2-sem; pre-registered 2026-10-29 13:52 PDT).** Adding an `all-MiniLM-L6-v2` sentence-embedding kernel to W2 changes the ALC against W2 by -0.0000 [-0.0003, +0.0003] on INV and by -0.0002 on A. Its gate, a CI-separated gain over W2 on INV, fails, so nothing was built.
- **Exact pair-rate posterior (W1).** No gain at budgets 1–3 (Section 4). We ran no confirmation and built no ZIP.

6 Limitations

- **The selection rests on simulation.** The real-draw effect (-0.0024, se 0.0016, 11 matched pairs) is post hoc, not significant, and lacks the simulated budget profile. The simulated populations come from the training release, whose subjects differ from the 2025–2026 competition subjects, and use random acquisition. The level change would cost +0.0018–+0.0046 ALC if the hidden pool were release-like.
- **Formative noise and feedback.** One draw has a standard deviation of about 0.0187 ALC. Arm means of 7 and 4 draws therefore cannot resolve an effect of the simulated size (-0.0019 to -0.0035). Choosing the level after formative feedback is mild tuning on the test pool, on the same 7 benchmark IDs in every run.
- **Statistics.** The CIs are percentile cluster bootstraps over as few as 20 benchmarks and can undercover. The populations overlap, and Table 1 is not adjusted for multiplicity. Our pre-registrations are self-hosted. The item layer's pre-registration and its results entered version control in the same snapshot, so their order rests on the times recorded in the files and logs.

- **Item information is local.** The layer learns only from labelled items of the same benchmark in the current run, and even with unlimited labels its signals explain a small share of the item-effect variance (Section 4). The cross-pair gain depends on how many pairs share a list. Item identity is an exact text match. Pairs with fewer than 80 items follow one assumed rule (Appendix D).
- **Scope of checks.** The local testbed checks engineering, not accuracy on frontier systems. The code, the experiments and this text were produced with an LLM coding assistant (Section 9). The audit checks that the text agrees with the result files, not that the analysis code is correct.

7 Conclusion

We submit 1019682, which combines cross-pair random effects on the global label list, a recalibrated budget-0 level and an online, within-benchmark item layer. Its expected gain over the base is small: -0.0019 to -0.0035 ALC across the simulated scenarios. The real draws neither confirm nor contradict that gain. The arm means (0.1904 and 0.1705) mostly reflect which pairs were drawn. A post-hoc pair-matched difference (-0.0024, se 0.0016) has the simulated size but not its budget profile. In simulation, an exact Bayes posterior does not improve the pair-rate component. Neither label-free item signal, an LLM's rating or sentence embeddings, helped.

8 Code and reproduction

- **Code, results and this report:** `https://github.com/neelsirivara/neurips2026-paiec`.
- **Submission:** `submission/candidates/w2_f05s1_k1q2.zip` (23.4 kB; members `README.md`, `labeling.py`, `model.py`), uploaded as Codabench submission 1019682. Its SHA-256, verified against our run records, is `d6d1c6a00e72b8aaf9e6628de6f03e50274a1d8f547883a0c0d70300aab969bf`.
- **Rerunning and rebuilding.** Uploading the ZIP unchanged reproduces a formative run; scores vary between runs, because each run draws new pairs. Appendix N gives all commands. The build steps alone rebuild both ZIPs in about a minute without gated data, and `submission/README.md` lists the expected hashes and ALCs.

9 Disclosure

- **Training sources.** `aims-foundations/measurement-db`, revision `54074453da`: the `subjects`, `items` and `responses` of the 163 competition-eligible directories, downloaded 2026-10-22 with the author's account. The organisers' benchmark inventory, used only to mark which release benchmarks are inventory rows. Both were used to simulate the protocol and select all constants, including the item layer's; the item studies used the release's item texts and features. No other data source was used, and the submitted files contain no data.
- **Formative feedback.** Aggregate scores of our runs guided the level change and screened the item layer, as the rules allow. We matched our own runs by anonymous pair IDs, but did not invert per-pair rows into success rates or try to identify subjects or benchmarks.
- **Pretrained models.** None in the selected submission (no `models.txt`, no network calls). In development only: Qwen2.5-Instruct 1.5B/3B/7B and Llama-3.2-3B-Instruct (Ollama) as local-testbed subjects; `Qwen/Qwen3-VL-4B-Instruct` to rate items and in three deployment probes; `Qwen/Qwen3.5-4B` in a speed comparison; `all-MiniLM-L6-v2` in W2-sem and one ablation; Qwen3-14B (4-bit, mlx) for a second rating study. Claude-API, Gemini and Groq rating routes were built but never run; no item was rated through any API.
- **Evaluation data.** GSM8K, SVAMP and ten BIG-Bench-Hard subtasks labelled the local models. These testbeds informed the few constants marked † in Appendix E, which remain in the submission's fallback path.
- **Reused code and compute.** Verbatim copies of the organisers' `check_submission_zip.py` and `streaming_ingestion.py` [Truong et al., 2026], used only for local checks. Everything ran on a CPU-only laptop, except the item ratings on a Mac GPU (3.2 h of model time for the Qwen3-VL study).
- **LLM-based coding assistant.** Claude Code (Anthropic's Claude models) was the coding and research assistant throughout. Under the author's direction it wrote the code, ran the experiments and analyses, and drafted this report, reading the project files (including training-release items and our Codabench logs). The author is responsible for the submission and the report.

References

N. E. Breslow and D. G. Clayton. Approximate inference in generalized linear mixed models. *Journal of the American Statistical Association*, 88(421):9–25, 1993. doi: 10.1080/01621459.1993.10594284.

Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950. doi: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.

Bradley Efron and Carl Morris. Data analysis using Stein's estimator and its generalizations. *Journal of the American Statistical Association*, 70(350):311–319, 1975. doi: 10.1080/01621459.1975.10479864.

Susan E. Embretson and Steven P. Reise. *Item Response Theory for Psychologists*. Psychology Press, 2000. doi: 10.4324/9781410605269.

Valentin Hofmann, David Heineman, Ian Magnusson, Kyle Lo, Jesse Dodge, Maarten Sap, Pang Wei Koh, Chun Wang, Hannaneh Hajishirzi, and Noah A. Smith. Fluid language model benchmarking. In *Conference on Language Modeling (COLM)*, 2025. arXiv:2509.11106.

Alex Kipnis, Konstantinos Voudouris, Luca M. Schulze Buschoff, and Eric Schulz. metabench – a sparse benchmark of reasoning and knowledge in large language models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.12844.

Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023. arXiv:2211.09110.

Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Routledge, 1980. doi: 10.4324/9780203056615.

David J. C. MacKay. The evidence framework applied to classification networks. *Neural Computation*, 4(5):720–736, 1992. doi: 10.1162/neco.1992.4.5.720.

Lorenzo Pacchiardi, Lucy G. Cheke, and José Hernández-Orallo. 100 instances is all you need: Predicting the success of a new LLM on unseen data by testing on a few instances, 2024. arXiv:2409.03563; presented at the KDD 2024 Workshop on Evaluation and Trustworthiness of Generative AI Models.

Yotam Perlitz, Elron Bandel, Ariel Gera, Ofir Arviv, Liat Ein-Dor, Eyal Shnarch, Noam Slonim, Michal Shmueli-Scheuer, and Leshem Choshen. Efficient benchmarking of language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024. arXiv:2308.11696.

Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: Evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.14992.

Pedro Rodriguez, Joe Barrow, Alexander Hoyle, John P. Lalor, Robin Jia, and Jordan Boyd-Graber. Evaluation examples are not equally informative: How should that change NLP leaderboards? In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4486–4503, 2021. doi: 10.18653/v1/2021.acl-long.346.

Yangjun Ruan, Chris J. Maddison, and Tatsunori Hashimoto. Observational scaling laws and the predictability of language model performance. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.10938.

Stanford AIMS Lab. Predictive AI evaluation competition (NeurIPS 2026). Competition web page, 2026. URL `https://aims.stanford.edu/competition`. Accessed 2026-10-21.

Sang Truong, Wale Salaudeen, Nhi Truong, Serena Wang, Angelina Wang, Luke Guerdan, Ziang Xiao, Xing Xie, Nick Haber, and Sanmi Koyejo. PAIEC predictor baselines. GitHub repository, 2026. URL `https://github.com/aims-foundations/paiec_baseline`.

Qinyuan Ye, Harvey Yiyun Fu, Xiang Ren, and Robin Jia. How predictable are large language model capabilities? A case study on BIG-bench. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. arXiv:2305.14947.

Yan Zhuang, Qi Liu, Zachary A. Pardos, Patrick C. Kyllonen, Jiyun Zu, Zhenya Huang, Shijin Wang, and Enhong Chen. Position: AI evaluation should learn from how we test humans. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2306.10512.

A Formative draws: details

All draws. Table 4 and Figure 2 list every formative draw of the base predictor (`xp_level_m050`: 1010717, the deployment probes 1010835 and 1010837, which return its predictions, and four re-uploads) and of the selected submission. Aggregates only, parsed from the saved Codabench Scoring output (`experiments/results/w2_items/draws.json`; no anonymous IDs are stored). The post-hoc pair-matched comparison of Section 5.2 averages each pair's scores within an arm and then differences the arms over the 11 pairs present in both; per budget it is -0.0114 (se 0.0028) at $n = 1$, +0.0013 at $n = 3$, +0.0009 at $n = 7$, -0.0012 at $n = 15$ and -0.0035 at $n = 31$. Matching by benchmark instead (7 shared benchmarks) gives -0.0209 (se 0.0151), still dominated by which pairs were drawn. *Not counted:* W2 run 1019686 was scored (8 pairs, log-only ALC 0.1581) but ended with `PAIEC-INFRA-ARCHIVE-001`, the platform's infrastructure archival error, and recorded no score; following the retry note added to `SWITCH_RULE.md` before the retry's result was retrieved, it was retried once with the same ZIP as 1019903, which is counted instead.

Table 4: Every formative draw used in the report (Codabench Scoring output; means over pairs). Base: `xp_level_m050`; probes: deployment probes returning the base predictions.

Run	Arm	Pairs	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC
1010717	base	8	0.2302	0.1832	0.1540	0.1433	0.1475	0.1416	0.1628
1010835	base (probe)	8	0.2425	0.1915	0.1963	0.1885	0.1775	0.1701	0.1920
1010837	base (probe)	9	0.2568	0.2438	0.2048	0.2039	0.1961	0.1893	0.2143
1019450	base	9	0.2450	0.2065	0.2056	0.1965	0.1702	0.1643	0.1967
1019451	base	8	0.2327	0.2066	0.1661	0.1708	0.1651	0.1561	0.1806
1019452	base	9	0.2412	0.1864	0.1710	0.1603	0.1617	0.1581	0.1758
1019453	base	9	0.2418	0.2187	0.2175	0.2121	0.1901	0.1860	0.2105
1019682	W2 (selected)	9	0.2528	0.2009	0.1833	0.1770	0.1711	0.1635	0.1881
1019683	W2 (selected)	9	0.1906	0.1861	0.1669	0.1599	0.1586	0.1565	0.1690
1019685	W2 (selected)	9	0.2266	0.1842	0.1551	0.1344	0.1266	0.1217	0.1549
1019903	W2 (selected)	9	0.2126	0.1833	0.1758	0.1571	0.1527	0.1506	0.1701

Table 5: Level constants, read from `model.py` inside each ZIP. Columns are Codabench submission IDs: 1010370 is our round-4 submission; 1010717 is the base of the selected submission 1019682, which keeps these constants; 1010715 and 1010735 are two other variants we scored.

Constant	Role	1010370	1010715	**1010717**	1010735
`SUBJECT_SHRINK`	weight of the hand-set subject terms	0.7	0	**0**	0
`PRIOR_INTERCEPT`	shift of the prior logit	0	0	$\mathbf{-0.5}$	-1
τ_μ (`XP_TAUS`)	prior sd of the global level μ	0.5	2	**2**	2

The level variants (October 26). Before the item layer, we compared single draws of four level variants (Table 6, Figure 3). The round-4 prior's budget-0 Brier (0.2834) exceeded the constant's 0.25; the recalibrated variants scored 0.2247–0.2568 at budget 0 on 5 draws, 1 of them above 0.25, and all had a lower budget-0 ECE (at most 0.278) than 0.340. Their ALC differences are within draw noise (largest benchmark-matched $|t| = 1.75$; Table 7).

Benchmark-matched differences. The anonymous benchmark IDs recur across runs (the page states that matching IDs denote the same benchmark); among the four level-variant runs no subject or pair recurred. For two runs we average each run's per-pair scores within each shared benchmark and take the mean and standard error over benchmarks of the difference (Table 7). Only aggregates are

Table 6: Formative runs of the level variants (October 26; means over pairs). Each run is a fresh draw on the same 7 hidden benchmarks, so rows are not paired comparisons. 1010835 and 1010837 are deployment probes returning 1010717's predictions. ECE: budget-weighted summary ECE; ECE_0: at budget 0.

Submission	Pairs	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC	ECE	ECE_0
1010370 (round 4, as shipped)	9	0.2834	0.2103	0.1909	0.1786	0.1641	0.1627	0.1934	0.169	0.340
1010715 (subject terms off)	9	0.2247	0.1732	0.1624	0.1532	0.1494	0.1449	0.1646	0.118	0.263
1010717 (selected)	**8**	**0.2302**	**0.1832**	**0.1540**	**0.1433**	**0.1475**	**0.1416**	**0.1628**	**0.142**	**0.278**
1010735 (stronger shift)	9	0.2458	0.1944	0.1971	0.1835	0.1740	0.1692	0.1913	0.117	0.219
1010835 (probe: 1010717's predictions, 2nd draw)	8	0.2425	0.1915	0.1963	0.1885	0.1775	0.1701	0.1920	0.124	0.228
1010837 (probe: 1010717's predictions, 3rd draw)	9	0.2568	0.2438	0.2048	0.2039	0.1961	0.1893	0.2143	0.142	0.254
1010717's predictor, mean of 3 draws	**–**	**0.2432**	**0.2062**	**0.1850**	**0.1785**	**0.1737**	**0.1670**	**0.1897**	**0.136**	**0.253**
constant 0.5 (any draw)	–	0.2500	0.2500	0.2500	0.2500	0.2500	0.2500	0.2500	–	–

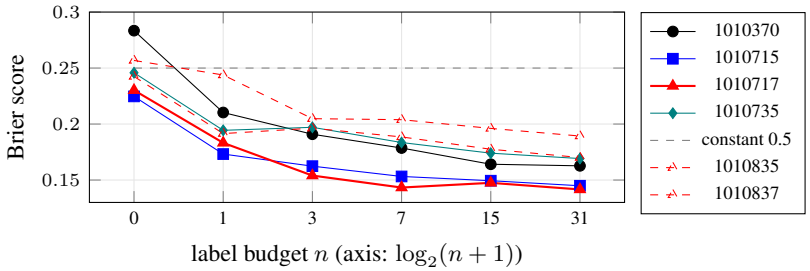


Figure 3: Formative learning curves of the level variants (Table 6). Red: 1010717's predictor on three draws (solid: 1010717; dashed: the probes).

stored (`experiments/results/deep_dive/formative_runs.json`); the parser checks that the ALC recomputed from the per-budget means equals the organisers' ALC (consistent).

Table 7: Benchmark-matched differences between formative runs: mean (standard error) over the 7 shared benchmarks. Negative favours the first run. In the ALC column no entry reaches two standard errors (largest $|t| = 1.75$). Of all 42 entries, 1 reaches two standard errors (largest $|t| = 2.42$), and 0 reach the two-sided 5% critical value of t with 6 degrees of freedom (2.45), before any correction for the 42 comparisons.

Runs	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC
1010715 − 1010370	-0.064 (0.033)	-0.054 (0.023)	-0.035 (0.025)	-0.041 (0.027)	-0.030 (0.030)	-0.033 (0.030)	-0.042 (0.024)
1010717 − 1010370	-0.045 (0.064)	-0.028 (0.034)	-0.032 (0.024)	-0.036 (0.027)	-0.017 (0.034)	-0.023 (0.033)	-0.029 (0.028)
1010735 − 1010370	-0.027 (0.068)	-0.029 (0.030)	-0.010 (0.023)	-0.012 (0.030)	-0.008 (0.032)	-0.010 (0.033)	-0.015 (0.029)
1010717 − 1010715	+0.019 (0.033)	+0.027 (0.041)	+0.002 (0.043)	+0.005 (0.042)	+0.014 (0.049)	+0.010 (0.044)	+0.012 (0.040)
1010735 − 1010715	+0.037 (0.059)	+0.026 (0.042)	+0.024 (0.040)	+0.029 (0.043)	+0.022 (0.044)	+0.023 (0.045)	+0.026 (0.041)
1010735 − 1010717	+0.018 (0.063)	-0.001 (0.022)	+0.022 (0.020)	+0.025 (0.026)	+0.009 (0.031)	+0.013 (0.031)	+0.014 (0.028)

Probe draws of the base predictor. The probe ZIPs' `model.py` begins with the base `model.py` (1010717) byte for byte and appends a wrapper that returns the base prediction; `labeling.py` is identical (verified; `experiments/results/deep_dive/selected_predictor_draws.json`, parsed from the saved Scoring output with `parse_feedback.py`). Their draws used the same 7 benchmark IDs as 1010717 and shared at most 1 pair with it. Benchmark-matched ALC differences between the draws are +0.0123 (se 0.0183; 1010835 − 1010717), +0.0230 (se 0.0159; 1010837 − 1010717) and +0.0107 (se 0.0145; 1010837 − 1010835). The largest of these differences between draws of one predictor exceeds 3 of the 6 ALC differences between submissions in Table 7, as expected if those are draw noise.

Budget-0 calibration in simulation. Budget-0 ECE (mean over pairs) of the previous predictor in four simulated populations: A 0.235, INV 0.271, HARD 0.299, SHIFT 0.286. The formative run of 1010370 gave 0.340. For the base's constants (1010717) the simulated values are A 0.257, INV 0.247 and HARD 0.203, and its formative run gave 0.278, also above every simulated value: the recalibrated level is closer, but probably still not right for the hidden pool. Per-pair ECE from about 62 evaluation items is noisy and biased upwards; the organisers report ECE only as a diagnostic. *A within-draw statistic.* $D = B_0 - B_{31}$ compares two budgets of the same draw. Simulated means for

the previous predictor are 0.082 (A), 0.095 (INV), 0.092 (MID) and 0.119 (HARD), with single-draw standard deviations 0.047–0.060; the formative value was 0.1207. Alone it separates the populations only weakly.

Table 8: Simulated populations (Table 1) with all three variants: ALC and difference to the previous predictor, mean [95% cluster-bootstrap CI over benchmarks]. Oracle: each pair's evaluation success rate as a constant. Source: `experiments/results/deep_dive/proto_level3.json`.

			1010715		1010717		1010735	
Population	Oracle	1010370	ALC	Δ [95% CI]	ALC	Δ [95% CI]	ALC	Δ [95% CI]
A	0.1731	0.1965	0.1968	+0.0002 [-0.0004, +0.0008]	0.1984	+0.0018 [+0.0005, +0.0032]	0.2022	+0.0057 [+0.0035, +0.0079]
Y25	0.1640	0.1860	0.1867	+0.0007 [-0.0008, +0.0022]	0.1906	+0.0046 [+0.0015, +0.0076]	0.1964	+0.0104 [+0.0058, +0.0148]
INV	0.1606	0.1887	0.1877	-0.0010 [-0.0021, +0.0002]	0.1868	-0.0020 [-0.0050, +0.0015]	0.1885	-0.0003 [-0.0053, +0.0054]
INV25	0.1692	0.1898	0.1883	-0.0015 [-0.0038, +0.0010]	0.1889	-0.0009 [-0.0063, +0.0052]	0.1917	+0.0019 [-0.0065, +0.0110]
MID	0.1664	0.1903	0.1892	-0.0011 [-0.0021, -0.0001]	0.1862	-0.0041 [-0.0057, -0.0025]	0.1861	-0.0042 [-0.0067, -0.0018]
HARD	0.1388	0.1630	0.1610	-0.0019 [-0.0036, -0.0002]	0.1561	-0.0069 [-0.0090, -0.0048]	0.1542	-0.0087 [-0.0115, -0.0057]
SHIFT	0.1731	0.2023	0.2017	-0.0006 [-0.0013, +0.0001]	0.1980	-0.0042 [-0.0056, -0.0028]	0.1968	-0.0055 [-0.0078, -0.0030]

Checks on the three variants. For each variant ZIP (1010717, 1010735, 1010715): the organisers' validator passes, passes, passes; the smoke test passes, passes, passes (44, 44, 44 cases); the stdlib `model.py` matches the numpy simulator with the same constants (max. difference 1.1×10^{-5}, 1.0×10^{-5}, 1.2×10^{-5} for 1010717, 1010735, 1010715), `labeling.py` is identical to the previous submission's, and the previous ZIP was unchanged by the build. Local-testbed ALC in the organisers' client: 0.2179 (1010717), 0.2180 (1010735), 0.2208 (1010715) and 0.2179 (1010370).

B Item layer (W2): details

Grids and selection. Category effects: $\tau_f \in \{0.25, 0.35, 0.5, 0.7\}$ and a share cap of distinct values per labelled item in $\{0.5, 1\}$ (a key whose values are mostly unique per item acts like an item identity, which the exact-item effect already models); kernel: $\tau_k \in \{0.5, 1, 2\}$, $q \in \{2, 4, 8\}$; plus the best category setting combined with the best kernel setting, 18 candidates in all. The criterion is the mean, over A, INV, MID and HARD on the tune folds, of the ALC difference to the base; per scenario the selected setting gives A -0.0015, INV -0.0010, MID -0.0018 and HARD -0.0020. Selected constants (read from the ZIP): $\tau_f = 0.5$, share 1, $\tau_k = 1$, $q = 2$. One code change after registration made the kernel's dot product loop over shared keys only (same arithmetic); the equivalence check below ran on the final code. The remaining settings of the grid were also run on the confirmation folds, after the gate was recorded, as exploratory results that cannot change the selection (`confirm.json`).

Checks on the ZIP. The organisers' validator passes; our smoke test passes; in the organisers' streaming client (mock coordinator, local testbed) the files run in stream mode with 0 `predict` calls during acquisition, ALC 0.2174 (base 0.2179). The stdlib `model.py` matches the numpy simulator to 1.5×10^{-5} on 57,945 predictions, and with both strengths at 0 it equals the base exactly. `labeling.py` is byte-identical to the base's.

W2-sem. Pre-registered in `PREREG_SEM.md` (written 2026-10-29 13:52 PDT, SHA-256 prefix `dbd4ff3fc845`) before any semantic variant was run. `all-MiniLM-L6-v2` embeddings of every item (computed offline on the laptop CPU) give a semantic kernel, alone or mixed with the character kernel, in a 504-setting grid tuned on folds 0–1. With infinite labels the semantic kernel explains 0.040 of the within-benchmark item-effect variance, about as much as the character kernel, and categories + character + semantic 0.210 against 0.184 without it (in-sample, slightly optimistic). On the confirmation folds the selected semantic variant changes the ALC against W2 by -0.0000 [-0.0003, +0.0003] on INV, -0.0002 [-0.0004, +0.0000] on A and -0.0003 [-0.0006, -0.0001] on MID. Its gate required a CI-separated gain over W2 on INV, which fails; it fails, and no ZIP was built.

W1. Development runs only (one dev seed; `experiments/results/deep_dive/w1/`). H7 mimics the hidden runs' structure as read from counts in our own logs (nine pairs on seven benchmarks, two benchmarks with two pairs, every subject used once, 120-item cap); H7-INV uses inventory benchmarks. No formative aggregate and no recovered pair rate set any value. Because no variant was worth confirming, no confirmation run was made and no ZIP built.

Exact pair-rate posterior (W1). These exploratory runs (one development seed) simulate the hidden runs' structure: about nine pairs on seven benchmarks, with no subject shared across benchmarks (configuration H7).

- An exact Bayes grid posterior over the benchmark level and pair deviation, with priors from the training folds, changes the ALC by +0.0017 [-0.0024, +0.0058] on H7-INV and by -0.0019 [-0.0035, -0.0003] on H7.
- Its H7 gain comes entirely from budget 0 (B_0 0.2472 vs. 0.2667), that is, from a different level.
- Updating faster after one label costs +0.0017 [+0.0002, +0.0033] on H7-INV.
- On a grid of prior sds, no setting beats the base's by more than 0.0002 in any of 4 scenarios.

Three probe uploads with the item-rating model in `models.txt` (1010835, 1010837, 1010865) indicate that it loads on the scoring GPU (Appendix C).

C Deep-dive analyses of the previous submission

All analyses use the round-4 simulator unchanged, configuration A unless stated (1,500 test sets).

Where the Brier is lost. The previous predictor reaches ALC 0.1965 against 0.1731 for the pair-rate oracle. The budget-weighted gap is +0.0074, +0.0079, +0.0052, +0.0027, +0.0009 and -0.0007 at $n = 0, \ldots, 31$ (total 0.0234), so 88% of it is at $n \leq 3$. Extreme pairs cost most: pairs with success rate below 0.1 have ALC 0.085 (oracle 0.030) and a budget-0 bias of +0.47; above 0.9, 0.098 (0.049) and -0.42. A single formative-sized test set has ALC standard deviation 0.039 in this configuration (one to three benchmarks per set).

What did not help. Re-tuning the cross-pair prior standard deviations on the inventory population (48 settings, nested by fold) changes the ALC by +0.0004 [-0.0007, +0.0014] on INV and +0.0003 on A. A subject-ability table keyed by `normalized_name` and learned on the training benchmarks has held-out within-benchmark correlation -0.06 to +0.10 with the pair rate ($R^2 \leq 0.019$).

Ceilings for side information. Table 9 adds oracle offsets to the previous predictor: the true benchmark mean logit (computed with the test pairs), and a fraction of each item's leave-pair-out effect estimated from all other release subjects. These are upper bounds for any difficulty estimate available at scoring time (for example from an item-rating model). They show that item-level difficulty, not the level, holds most of the remaining headroom.

Table 9: Simulated ALC with oracle side information inside the previous predictor (A: release-like, HARD: mean rate ≤ 0.4); difference to the previous predictor (A: 0.1965, HARD: 0.1630) with 95% cluster-bootstrap CI.

	A		HARD	
Oracle offset	ALC	Δ [95% CI]	ALC	Δ [95% CI]
benchmark mean logit (with test pairs)	0.1900	-0.0065 [-0.0081, -0.0051]	0.1516	-0.0114 [-0.0148, -0.0079]
item effect from other subjects, $\times 0.5$	0.1655	-0.0310 [-0.0348, -0.0272]	0.1364	-0.0266 [-0.0331, -0.0208]
item effect from other subjects, $\times 1$	0.1548	-0.0417 [-0.0475, -0.0360]	0.1287	-0.0342 [-0.0438, -0.0259]
benchmark mean + item effect	0.1494	-0.0471 [-0.0526, -0.0417]	0.1193	-0.0437 [-0.0530, -0.0353]

LLM-rated item difficulty: design. In simulation most of the remaining headroom is item difficulty (Appendix C), so we tested whether a small open LLM can supply it at scoring time. *Pre-registration.* `PREREG_items.md` (written 2026-10-26 13:08 PDT, SHA-256 prefix `06d9e784369b`) fixed the signal, items, offset model and gate before any item was rated. Three amendments followed, each before any rating was analysed: Amendment 1 (2026-10-26 13:18 PDT) let the rating model be chosen between two 4B models by speed alone; Amendment 2 (2026-10-26 13:31 PDT) added an intercept-only control, after a dry run on stub ratings showed that the fitted intercept alone moves the ALC; Amendment 3 (2026-10-26 14:08 PDT) registered an API rating route with the same analysis, which was built and never run. *Signal.* Qwen/Qwen3-VL-4B-Instruct (snapshot `ebb281ec70b0`), run locally, read each item and answered whether a state-of-the-art 2025–26 AI system would most likely complete it; the signal is $s_1 = \log P(\text{Yes}) - \log P(\text{No})$ from one forward pass without generation. It rated all 24,199 items (20,720 distinct prompts) of two populations: INV (33 inventory

benchmarks, the proxy for the hidden pool) and AS (25 seeded non-inventory benchmarks, a release-like check). *Gate.* The candidate adds $w\,z(s_1) + b$ to the prior offset of 1010717, with w and b fitted leave-benchmark-out in each fold. It ships only if (a) on INV its ALC difference has a 95% cluster-bootstrap CI below 0 and (b) on AS it is not significantly worse, both against the base and against the intercept-only control.

The gate fails (Table 10): (a) fails in both comparisons, (b) holds. Against the base the ALC changes by +0.0005 [-0.0015, +0.0024] on INV and -0.0023 [-0.0050, +0.0008] on AS; against the control by -0.0006 [-0.0014, +0.0002] on INV. The signal is real but weak. Its slope is positive in 5 of 5 folds (0.15–0.19 per standard deviation), so unlike our round-3 text prior its sign transfers across benchmarks. But its within-benchmark correlation with the empirical item effect is only 0.089 on INV and 0.037 on AS (medians over benchmarks). The differences to the base are mostly the level effect of b at budget 0 (+0.0080 on INV, -0.0252 on AS), which the control isolates. The rating route was therefore dropped; no submitted ZIP contains a model. We still checked deployment, with three uploads that add the rating model to `models.txt` of the base ZIP (1010835, 1010837, 1010865). Probes 1010835 and 1010837 return the base predictor's predictions, so their scores are two extra draws of the base (Tables 6 and 4). Probe 1010837 finished with a valid score, which by its design means that the model loaded from the prefetched cache and ran a forward pass on the scoring GPU, unless the probe did not run at all. Probe 1010865 stopped with the platform's invalid-output error (`PAIEC-PREDICT-002`), which it returns only after a successful load on a GPU of at least 70 GB.

LLM-rated item difficulty: details. Table 10 gives every comparison of the study in Section 5.3 (source: `experiments/results/deep_dive/items/eval_items_qwen3vl_4b.json`, which records the hashes of the pre-registration and of Amendments 1 and 2, consistent with the files; ratings file SHA-256 prefix `390442e88612`). Every needed item was rated (coverage 100% on INV and 100% on AS; 0.56 s per prompt on average). The fitted maps per fold have slope w from 0.15 to 0.19 and intercept b from 0.33 to 0.56, close to the intercept-only control's 0.32–0.53; on INV the rating changes B_1–B_{31} by at most 0.0006. The within-benchmark correlation of s_1 with the leave-pair-out empirical item effect has median 0.089 (mean 0.109, positive on 77% of 26 benchmarks) on INV and median 0.037 (mean 0.033, 67% of 21) on AS. The exploratory signal s_3, the mean log-probability of the item text, and the combination $s_1 + s_3$ also have CIs that include 0 on INV. Amendment 3 (SHA-256 prefix `63ca97b978f9`) registered rating by the Claude API with a verbalised probability; the script exists, but no API rating was ever made, so that candidate has no result.

Table 10: Pre-registered item-difficulty study (Qwen/Qwen3-VL-4B-Instruct, $s_1 = \log P(\text{Yes}) - \log P(\text{No})$): ALC of each variant, its difference to the base (1010717; INV 0.1868, AS 0.1950) with 95% cluster-bootstrap CI over benchmarks, and the budget-0 part of that difference. The last row is the Amendment-2 comparison against the intercept-only control. Gate: (a) INV CI below 0 fails against the base and fails against the control; (b) AS not significantly worse holds.

	INV (33 inventory benchmarks)			AS (25 release-like benchmarks)		
Comparison	ALC	Δ [95% CI]	ΔB_0	ALC	Δ [95% CI]	ΔB_0
s_1 (pre-registered) − base	0.1873	+0.0005 [-0.0015, +0.0024]	+0.0080	0.1928	-0.0023 [-0.0050, +0.0008]	-0.0252
intercept only (control) − base	0.1879	+0.0011 [-0.0011, +0.0030]	+0.0092	0.1928	-0.0022 [-0.0045, +0.0003]	-0.0219
s_3 (exploratory) − base	0.1876	+0.0008 [-0.0016, +0.0030]	+0.0081	0.1933	-0.0018 [-0.0044, +0.0011]	-0.0212
$s_1 + s_3$ (exploratory) − base	0.1871	+0.0003 [-0.0018, +0.0024]	+0.0074	0.1932	-0.0018 [-0.0048, +0.0013]	-0.0224
s_1 − intercept only	–	-0.0006 [-0.0014, +0.0002]	–	–	-0.0000 [-0.0010, +0.0011]	–

D Round 4: details

Protocol. The round-4 simulator rebuilds the official protocol on the training release with five benchmark-held-out folds. A *test set* is a set of pairs of one fold sharing one global list. Per pair, items are split 50/50 and the acquisition pool is permuted; at budget n every prediction of the test set receives the union of all its pairs' first n labels. The primary configuration A draws 1–3 random benchmarks and 10 pairs with 100 items each ($\leq$1,000 subject–item pairs, as in formative runs), 300 test sets per fold. Configurations B–E vary the composition (paragraph *Configurations* below). The design, the grid of 864 prior settings, nested selection and a ship rule were pre-

Table 11: Round-4 simulation on the training release: Brier ALC by test-set configuration, units (test set, pair), 95% cluster-bootstrap CIs over benchmarks. S0: our rounds 1–3 per-pair predictor as the evaluator runs it; S1: the same with the target pair's labels only; XP: cross-pair model (nested selection; level constants of 1010370); oracle: each pair's evaluation success rate as a constant.

Configuration	units/bench.	S0	S1	XP	oracle	XP − S0 [95% CI]	XP − S1 [95% CI]
A: formative-like (primary)	13,460/135	0.2095	0.2074	0.1966	0.1731	-0.0129 [-0.0151, -0.0109]	-0.0108 [-0.0134, -0.0084]
E: distinct models	8,673/135	0.2078	0.2074	0.1953	0.1730	-0.0125 [-0.0148, -0.0100]	-0.0121 [-0.0146, -0.0095]
B: one benchmark, all pairs	6,772/135	0.2131	0.2087	0.1842	0.1793	-0.0289 [-0.0342, -0.0239]	-0.0245 [-0.0312, -0.0181]
C: one fold, many benchmarks	9,577/125	0.2190	0.2084	0.1797	0.1780	-0.0393 [-0.0521, -0.0280]	-0.0287 [-0.0384, -0.0191]
D: A + eval-only small pairs	14,293/135	0.2097	0.2096	0.1969	0.1706	-0.0128 [-0.0150, -0.0108]	-0.0127 [-0.0160, -0.0094]

registered (`experiments/results/round4/PREREG.md`, SHA-256 prefix `b9e95a5b47e9`) before any outcome was computed.

Cross-pair gain. Table 11. In configuration A the cross-pair model reaches 0.1966 against 0.2095 for the per-pair predictor (-0.0129 [-0.0151, -0.0109], lower on 73% of units), closing 35% of the gap to the pair-rate oracle. The gain holds when cross-pair evidence only comes from different models (E: -0.0125 [-0.0148, -0.0100]) and grows when more pairs share the list (B: -0.0289; C: -0.0393). Removing the benchmark effect costs +0.0048 [+0.0032, +0.0064] and removing the exact-item effect +0.0048 [+0.0038, +0.0059]: benchmark difficulty, invisible at budget 0, and item difficulty, invisible from a pair's own 31 labels, contribute about equally (Table 13). The fresh-first rule changes the ALC by -0.0005 [-0.0009, -0.0002], almost all at $n \geq 15$.

Inside the organisers' client. Driven by a mock coordinator on the local testbed, the organisers' `run_streaming` runs the base files (1010717) in `stream` acquisition mode with 0 `predict` calls during acquisition and 31 labels on each of the 17 pairs; the ALC is 0.2179 against 0.2179 for the previous files (the testbed's small 2024 models hardly feel the level change). The base `model.py` matches the vectorised simulator to 1.1×10^{-5}, and the organisers' validator and our smoke test pass (Appendix A).

Table 12: Brier score per budget and ALC (configurations A and E) and the paired difference XP − S0 with its 95% cluster-bootstrap CI (level constants of 1010370).

	Method	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC
A	S0 (rounds 1–3, as deployed)	0.2475	0.2221	0.2098	0.2000	0.1956	0.1924	0.2095
A	S1 (own pair's labels only)	0.2475	0.2284	0.2107	0.1957	0.1876	0.1819	0.2074
A	XP (cross-pair, shipped)	0.2475	0.2126	0.1992	0.1868	0.1776	0.1663	0.1966
A	oracle: pair rate	0.1731	0.1731	0.1731	0.1731	0.1731	0.1731	0.1731
A	XP − S0	–	-0.0096	-0.0106	-0.0132	-0.0180	-0.0261	-0.0129
	95% CI	–	[-0.0131, -0.0063]	[-0.0133, -0.0082]	[-0.0156, -0.0109]	[-0.0211, -0.0151]	[-0.0301, -0.0224]	[-0.0151, -0.0109]
E	S0 (rounds 1–3, as deployed)	0.2487	0.2282	0.2104	0.1958	0.1886	0.1831	0.2078
E	S1 (own pair's labels only)	0.2487	0.2294	0.2106	0.1950	0.1871	0.1812	0.2074
E	XP (cross-pair, shipped)	0.2487	0.2135	0.1976	0.1846	0.1751	0.1628	0.1953
E	oracle: pair rate	0.1730	0.1730	0.1730	0.1730	0.1730	0.1730	0.1730
E	XP − S0	–	-0.0147	-0.0128	-0.0111	-0.0135	-0.0203	-0.0125
	95% CI	–	[-0.0207, -0.0087]	[-0.0161, -0.0094]	[-0.0135, -0.0085]	[-0.0167, -0.0102]	[-0.0250, -0.0154]	[-0.0148, -0.0100]

Configurations. **A** (primary, formative-like): 1–3 random benchmarks, 10 random pairs with at least 80 items split over them, 100 items per pair, 300 test sets per fold; **E**: as A but no two pairs share a model name, so cross-pair evidence comes only from different models; **B**: all pairs of one benchmark (up to 300, 200 items each); **C**: up to 2,000 pairs of a whole fold in one list; **D**: A plus up to two pairs with 20–79 items that are evaluated on all items and never acquired (the page does not say how such pairs are handled). Per-pair ALC; means over (test set, pair) units. Units, benchmarks, test sets and labels at budget 31: A 13,460/135/1,500/417,260; E 8,673/135/1,500/268,863; B 6,772/135/135/209,932; C 9,577/125/5/296,887; D 14,293/135/1,500/417,260. In D the acquired pairs gain -0.0129 [-0.0151, -0.0109] and evaluation-only pairs -0.0120 [-0.0217, -0.0036]. Frame structure: median 17.5 pairs and 3 model names per benchmark (maximum 4,278 pairs), 342 names on two or more benchmarks; 98.5% of labels are on an item that at least two pairs answered. Each pair keeps its 200 answered items with the lowest benchmark-level hash rank, so subjects of a benchmark share items as far as their answered sets allow.

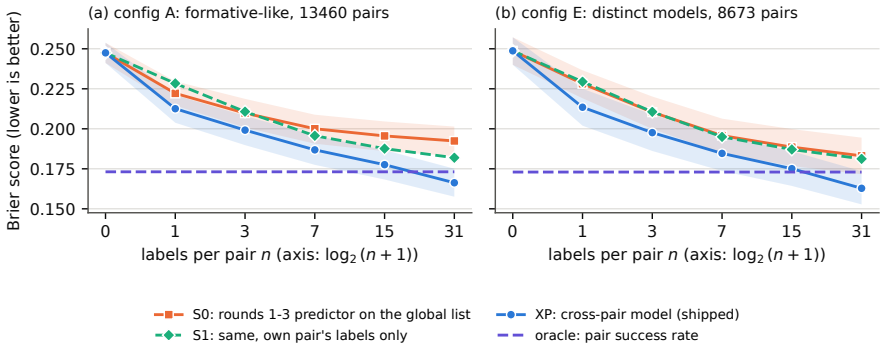


Figure 4: Brier score per budget on the training release (benchmark-held-out). Bands: marginal 95% cluster-bootstrap CIs over benchmarks; the paired differences, which are much tighter, are in Table 12. XP falls below the pair-rate oracle at $n = 31$ because it learns item difficulty from other subjects.

Table 13: Components (all-fold setting; configurations A and E): ALC and paired difference to the shipped setting (positive: removing the component hurts).

	A: formative-like		E: distinct models	
Variant	ALC	Δ [95% CI]	ALC	Δ [95% CI]
shipped setting (all effects)	0.1965	–	0.1952	–
− benchmark effect	0.2013	+0.0048 [+0.0032, +0.0064]	0.2003	+0.0050 [+0.0032, +0.0070]
− subject effect	0.1969	+0.0004 [+0.0001, +0.0008]	0.1954	+0.0002 [-0.0002, +0.0006]
− subject–benchmark effect	0.1975	+0.0010 [+0.0004, +0.0017]	0.1959	+0.0006 [-0.0000, +0.0014]
− exact-item effect	0.2013	+0.0048 [+0.0038, +0.0059]	0.2005	+0.0053 [+0.0038, +0.0068]
− probit integration	0.1970	+0.0005 [+0.0000, +0.0009]	0.1958	+0.0006 [-0.0000, +0.0011]
benchmark + pair effects only	0.2018	+0.0053 [+0.0043, +0.0063]	0.2008	+0.0056 [+0.0041, +0.0071]
name/subject + pair effects only	0.2095	+0.0130 [+0.0105, +0.0157]	0.2095	+0.0143 [+0.0114, +0.0173]
pair + item effects only	0.2101	+0.0135 [+0.0099, +0.0174]	0.2096	+0.0144 [+0.0100, +0.0189]
pair effect only	0.2167	+0.0202 [+0.0163, +0.0243]	0.2168	+0.0216 [+0.0171, +0.0264]

Table 14: Runtime of the round-4 `model.py` on one laptop core (synthetic global lists of 250-word items, 31 labels and 50 evaluation items per pair): first call of a checkpoint (parse and fit), later calls, and the whole checkpoint. The selected file differs only in constants.

Pairs	Labels	Predictions	Fit (s)	Per call (ms)	Checkpoint (s)
10	310	500	0.01	0.102	0.1
100	3,100	5,000	0.07	0.094	0.5
1,000	31,000	50,000	0.56	0.098	5.5
2,000	62,000	100,000	1.68	0.116	13.3
4,000	124,000	200,000	3.68	0.128	29.3

Pre-registration and selection. `PREREG.md` (consistent with every result file) fixed configurations A–D, predictors, the grid of 864 settings (τ_b, τ_{name}, τ_s, $\tau_{\text{name},b}$, τ_{sb}, τ_d, probit on/off; $\tau_\mu = 0.5$), nested selection (for each outer fold, the best setting on the other four folds' config-A test sets), the primary outcome and the ship rule: ship iff (a) the config-A ALC difference to S0 has a CI below zero, (b) the real testbed is not significantly worse, (c) no other configuration is significantly worse, and (d) engineering gates hold (never raises, smoke test and validator pass, a 62,000-label checkpoint in $\leq$60 s). Rule (a) holds, (b) holds, (c) holds. The model also has slots for a model-name effect and a name–benchmark effect, which the selection switched off. The selected setting lay on the grid boundary in τ_b and τ_d; an addendum written after the primary result (consistent) added configuration E and an extended grid of 2,160 settings, selected on A and E jointly; it returned the same setting. The stdlib `model.py` and the vectorised simulator agree to 1.4×10^{-5} on 56,750 predictions.

Results in more detail. Per budget (Table 12, Figure 4), budget 0 is unchanged (the list is empty); the difference is significant at every budget from $n = 1$ (-0.0096 [-0.0131, -0.0063] in A) and largest at $n = 31$ (-0.0261 [-0.0301, -0.0224]), where XP (0.1663) is below the pair-rate oracle (0.1731). Against S1, i.e. a correct per-pair filter, the gains are -0.0108 (A) to -0.0287 (C). Components (Table 13): the subject–benchmark effect (+0.0010), the subject effect (+0.0004) and probit integration (+0.0005) are small; the subject effect rarely transfers because full attribute dictionaries rarely repeat across benchmarks (211 of 16,378 do). Benchmark and pair effects alone recover more than half of the gain (0.2018 against 0.2095 for S0), whereas item and pair effects alone recover none (0.2101): the item effect becomes useful only together with the benchmark effect, which puts the other subjects' labels on a common scale. The best ALC per τ_d is 0.2013, 0.1969, 0.1965, 0.1968, 0.1980 for $\tau_d = 0, 1.5, 2.25, 3, 4$ (A). Real testbed (seed 0): XP 0.2222 against 0.2295 for the per-pair predictor as deployed; against a correct per-pair filter -0.0002 [-0.0072, +0.0074]; at $n = 1$ XP is worse there (0.2549 vs. 0.2468). Fresh-first on the real testbed: +0.0001 [-0.0010, +0.0011] (not significant). If the evaluator passed only the target pair's labels, XP would still be slightly better on the release (-0.0016 [-0.0028, -0.0005] against S1) but worse on the real testbed (+0.0060 [+0.0022, +0.0099]).

Selection. The nested selection picked the all-fold setting in 3 of 5 outer folds; the per-fold config-A differences XP $-$ S0 range from -0.0167 to -0.0074 (negative in 5 of 5). All-fold setting (shipped): $\tau_\mu = 0.5$, $\tau_b = 1.5$, $\tau_{\text{name}} = 0$, $\tau_s = 0.5$, $\tau_{\text{name},b} = 0$, $\tau_{sb} = 0.6$, $\tau_d = 2.25$, probit on; with it the config-A difference to S0 is -0.0130 [-0.0151, -0.0110] (nested estimate: -0.0129).

Amendment and addendum. One amendment was made to `PREREG.md` before any outcome was computed: the acquisition rule's skip threshold (registered as labels_remaining/items_remaining $<$ 0.5, which with 50-item pools would never skip) was replaced by the minimal one that still spends the budget. Addendum 1 was written at 20:58 PDT after the primary result and before any further outcome: configuration E, the extended grid ($\tau_b \in \{0.5, 1, 1.5, 2, 3\}$, $\tau_d \in \{0, 0.75, 1.5, 2.25, 3, 4\}$), and a "robust pick" minimising the average of the A and E ALCs, to be shipped only if not significantly worse than the pre-registered pick on A and better than S0 on E. It selected the same setting (5 of 5 folds; nested difference to the pre-registered nested XP on A -0.0001 [-0.0002, +0.0000]); shipped: robust pick (addendum 1). Best ALC per τ_d on E: 0.2005, 0.1979, 0.1957, 0.1952, 0.1955, 0.1964 for $\tau_d = 0, 0.75, 1.5, 2.25, 3, 4$.

Checks. Pre-registered equivalence: 24 config-A test sets re-predicted by the stdlib `model.py` from dictionary inputs with binary labels: maximum absolute difference 1.4×10^{-5}. Real-testbed per-budget Brier (seed 0) for XP / S0-deployed / per-pair: B_1 0.2549/0.2468/0.2472, B_7 0.2100/0.2219/0.2118, B_{31} 0.1916/0.2130/0.1991. Fresh-first skipped 154,939 candidates in configuration A (ALC 0.1960 vs. 0.1965).

E Per-pair predictor (rounds 1–3; now the fallback and the budget-0 prior)

All constants below are hand-set and are still in `model.py`: the budget-0 prior is the offset of the cross-pair model, the rest is its fallback. Values are those of the round-4 file (1010370); the base 1010717 of the selected submission 1019682 differs only in `SUBJECT_SHRINK` (0 instead of 0.7, which removes the whole subject part below) and `PRIOR_INTERCEPT` (-0.5 instead of 0), besides τ_μ of the cross-pair model (Table 5). The few whose values were adjusted after looking at the local testbeds are marked † and have sensitivity analyses in Appendix G.

Budget-0 prior. A logistic prior $p_0 = \sigma(z)$ with $z = \mathrm{clip}(z_{\mathrm{subj}} + z_{\mathrm{item}}, \pm 1.5)$, so $p_0 \in [0.18, 0.82]$. The subject part is $z_{\mathrm{subj}} = 0.7 \cdot (0.35\,\mathrm{clip}(t - 2024.5, \pm 2) + e + 0.25\,\mathrm{clip}(\log_2(N/30\mathrm{B}), \pm 4) + 0.1\,\mathbb{1}[\text{non-default harness}])$, where t is the release year (fractional), $e \in \{0, 0.15, 0.3, 0.45\}$ encodes an empty, low, other or high `reasoning_effort`, and N is a parameter count parsed from the name ("7b", "70B"); when no count is present, name tokens such as *mini/nano/flash* contribute -0.3 and *pro/opus/ultra* contribute 0.2. A per-provider table (0 entries, inside the multiplier) and an intercept δ added after it (`PRIOR_INTERCEPT`: 0 in round 4, -0.5 in the selected submission) complete the subject part; the provider table is empty and is the slot a fitted coefficient table would fill. The item part parses `key=value` pairs in `item_features`: a tier/difficulty/level value matching *hard/expert/...* adds -0.6, *easy/basic/...* adds 0.5, a numeric level $\ell \in [1, 5]$ adds $-0.25(\ell - 3)$, and points above one subtract 0.1 each (capped at five); finally the item length adds $-0.12\,\mathrm{clip}(\log_2(L/600), \pm 3)$ for L characters. The directions encode common knowledge (newer, larger, reasoning-enabled subjects do better; harder, longer, higher-value items are failed more often); the magnitudes were chosen to move the prior by at most a few tenths of a logit per attribute and were not tuned on either testbed.

Pair-level shrinkage. Given n acquired labels $y_1, \ldots, y_n$ of the same pair with item logits $z_{\mathrm{item}}^{(j)}$ and mean $\bar{z}$, the prior rate is $r = \sigma(\mathrm{clip}(z_{\mathrm{subj}} + \bar{z}))$ and the posterior mean of a Beta–binomial model with prior strength $\alpha_0 = 4^{\dagger}$ pseudo-counts is $\rho = (\alpha_0 r + \sum_j y_j)/(\alpha_0 + n)$ [Efron and Morris, 1975]. The pair estimate for a target item is $p_{\mathrm{pair}} = \sigma(\mathrm{logit}\,\rho + z_{\mathrm{item}} - \bar{z})$, i.e. the item cues shift the pair rate on the logit scale.

Item-level component 1: ridge features. From $n \geq 7$ labels, three cheap features of the item text—log length, digit density and log(1+number of sentences)—are standardised over the acquired items and a logistic regression with offset $\mathrm{logit}\,p_{\mathrm{pair}}$ and ridge penalty $\lambda = 4$ is fitted by six Newton steps; its adjustment $\mathrm{clip}(\beta^{\top}\tilde{f}, \pm 1)$ is added to the target's logit.

Item-level component 2: TF-IDF nearest neighbours. Item texts (content, features and interactors, truncated to 4000 characters) are tokenised into lower-case words and numbers; the acquired set defines the vocabulary and document frequencies, with sublinear term frequency, smoothed idf and ℓ_2 normalisation (bigram weight 1; target terms absent from the acquired vocabulary are dropped: no). With cosine similarities s_j between the target and acquired item j and weights $w_j = (\kappa s_j)^2$, the correction is a shrunken kernel-weighted residual,

$$p = \mathrm{clip}\Big(a + \frac{n}{n + N_0} \cdot \frac{\sum_j w_j\,(y_j - a)}{\tau + \sum_j w_j}\Big), \qquad N_0 = 4,\ \tau = 2, \tag{4}$$

where the anchor a is the pair estimate (optionally including the ridge adjustment) and $\kappa = 2$ by default. With at most 31 short documents most bigrams are unique, so cosine similarities are small (typically below 0.15 even for related items), which deliberately keeps the correction small unless items share many terms; Appendix H compares a sharper unigram-only representation.

Leave-one-out gating. From $n \geq 5$ labels, every configuration in $\{\text{ridge off, on}\} \times \{\kappa \in \{0, 1, 2, 4\}\}$ ($\kappa = 0$ means no kNN) is scored by its leave-one-out Brier score on the acquired labels themselves: label j is predicted from the other $n - 1$ labels, including a leave-one-out shrinkage estimate and a leave-one-out ridge fit. The best configuration replaces plain shrinkage only if its mean per-item improvement exceeds 2 standard error(s) of that mean†; otherwise the predictor returns p_{pair}. This makes adaptation *opt-in per pair and budget*: a pair whose labels show no exploitable structure is predicted by shrinkage alone. Below 5 labels no item-level component is used at all (in preliminary runs an always-on kNN from one or three labels only added noise at B_1 and B_3).

Acquisition policy. `labeling.py` implements the streaming interface and never raises. Candidates are accepted in stream order (uniformly random, hence the same label distribution as the evaluator's default) except that a candidate whose maximum TF-IDF cosine similarity to an already acquired item exceeds 0.6 is skipped while fewer than 90% of the remaining candidates are needed; it always accepts when the remaining candidates are needed to spend the budget. We compare this *novelty* filter with the evaluator's random policy, plain stream order, the organisers' uncertainty band $|p - 0.5| \leq 0.15$ [Truong et al., 2026], and a novelty-plus-uncertainty variant (skip $|p - 0.5| > 0.45$ once 7 labels are known).

Implementation (round-4 and selected files). Predictions are clipped to $[0.01, 0.99]$; `predict` is wrapped in a fallback chain (cross-pair model → per-pair full model → shrinkage only → smoothed label mean → 0.5), accepts labels given as booleans, numbers or strings, tolerates missing fields, malformed entries and non-dictionary inputs, and is deterministic. The round-4 ZIP (1010370; 19.8 kB; members `README.md`, `labeling.py`, `model.py`) contains no `models.txt` and no `requirements.txt`: both files import only standard-library modules (`model.py`: `array`, `base64`, `hashlib`, `json`, `math`, `os`, `re`, `sys`, `threading`, `zlib`; the round-3 item-text-prior hooks stay inactive because no weight file is shipped; `labeling.py` loads its sibling `model.py` by path) and make no network calls. The organisers' `check_submission_zip.py` (commit `cc88a959`) passes on the ZIP. Our smoke test calls `predict` with the organisers' sample inputs at every budget and with malformed labels and inputs (44 cases), plus 220 cross-pair cases, and checks determinism; with the cross-pair model enabled the mean latency is 0.32 ms per call on short items and 0.06 ms at budget 31 with 300-word items (maximum 0.7 ms), about 16,449 calls per second on one laptop core.

F Rounds 1–3: data, testbeds and the training release

What we could and could not use. The official training data (`measurement-db`: item-level responses of many subjects on curated benchmarks) require accepting terms with a Hugging Face account. For most of the project we had no credentials and did not try to obtain the data in any other way; the per-pair predictor and all its constants were fixed before any of it was seen. Its public file listing (2026-10-22, revision `54074453da`) held 281 benchmark directories: 14.8 GB of `benchmarks`, `subjects`, `items` and `responses` tables plus 107.8 GB of traces and 154.7 GB of assets. On that day the author's login became available and we downloaded the 163 competition-eligible directories (binary, item-level responses; the eligibility flags live inside the `benchmarks` tables) without traces or assets, and used them as described next. We also inspected the organisers' inventory of eligible benchmarks (1,261 entries) but fitted nothing on any of them. The two local testbeds below only evaluate design choices; the few constants they influenced are marked in Appendix E and probed in Appendix G.

The training release: validation and a fit that was not shipped. `fit_prior_from_measurement_db.py` builds one row per (subject, test condition, item) with the mean response over trials as label—16,718 pairs with at least 20 distinct items from 144 benchmarks, 3,118,742 rows (pairs capped at 200 items; 16,303 pairs have the 80 items the protocol requires)—and does two things (Appendix J). First, it evaluates the rounds 1–3 prior-plus-shrinkage rule with the protocol in closed form (50/50 split, 31 random acquisitions, Eq. 1; without the item-level components, which change the ALC by < 0.001 locally): ALC 0.2051 [0.2042, 0.2060] against 0.2396 [0.2380, 0.2411] for the organisers' empirical mean and 0.2470 for the constant, with $\alpha_0 = 4$ the best of $\{2, 4, 8\}$. The ordering and the size of the gain measured locally thus transfer to real systems on real benchmarks. Second, it fits an L_2-penalised logistic regression on exactly the feature terms of the hand-set prior plus a provider table and benchmark fixed effects (nuisance parameters; the shipped intercept is their mean), pairs weighted equally, with five-fold cross-validation over whole benchmarks. The fitted prior *loses* to the hand-set one on held-out benchmarks at every penalty (budget-0 Brier 0.2421–0.2455 against 0.2388; paired differences at the selected setting +0.0033 [+0.0030, +0.0036] at budget 0 and +0.0009 [+0.0008, +0.0009] in ALC), and an intercept-only prior is worse still (0.2519). The ship-only-if-better rule therefore kept the hand-set constants; in rounds 1–3 no coefficient of the submission derived from the release (round 4 selects the cross-pair model's prior standard deviations on it, Appendix D). Where the data speak, the fitted signs agree with the hand-set values; what held-out benchmarks punish is the provider table (Appendix J).

An item-text difficulty prior (pre-registered; not shipped). Benchmark difficulty dominates the pair rate (benchmark means have standard deviation 0.223) and the attribute prior cannot see it. We therefore tested, with design and ship rule fixed in `PREREG.md` (SHA-256 `bb54af425d…`) before any fit, whether the item text predicts difficulty on *unseen* benchmarks: a ridge regression on 262,144 signed-hashed buckets (words, bigrams, character trigrams, `item_features`) and 16 text statistics (length, LaTeX, code, answer choices, …), trained on the item logit the hand-set prior leaves unexplained (109,233 items, benchmarks weighted equally) and stacked onto the prior as $T = Ws + B$ with W, B fitted on out-of-fold scores; shrinkage is unchanged. On the same

benchmark-held-out folds, every choice nested inside the training benchmarks, the ALC changes by -0.0000 [-0.0013, +0.0014] (cluster bootstrap over 135 benchmarks), closing -0.1%, 0.1% and 0.1% of the gap to the pair-rate oracle at $n = 0, 1, 3$. The inner CV chose the largest penalty of the grid in 5 of 5 folds and the out-of-fold slope W was *negative* in all 5 (-0.32 to -0.03): text patterns learned on some benchmarks point the wrong way on others. Exploratory variants registered afterwards (penalties up to 10^4; statistics alone) give +0.0002 and +0.0001, the local testbeds +0.0019 (real, not significant) and +0.0003 (synthetic, not significant). Headroom exists—knowing each held-out benchmark's mean rate would gain -0.0061 ALC—but item text in this form does not reach it; the rule fails and the submission is unchanged (Appendix K).

Real Ollama testbed. To obtain item-level correctness labels of real language models we ran four small open-weight models locally with Ollama (Q4_K_M quantisation, greedy decoding, CPU only): Qwen2.5-Instruct 1.5B, 3B and 7B and Llama-3.2-3B-Instruct. Three public benchmarks were used, none of which appears in the PAIEC inventory (only a derived variant, `mr_gsm8k`, does): the first 200 GSM8K test items, the first 200 SVAMP test items (with the SVAMP problem type as `item_features`), and a 300-item mix of ten BIG-Bench-Hard subtasks (30 items each, subtask name as `item_features`). Two harnesses are represented: a direct-answer prompt ("reply with only the final answer"; exact match after numeric or option normalisation) and, for GSM8K/SVAMP, an earlier 8-shot chain-of-thought run of the Qwen models whose greedy answers we re-used. Each (model, harness, benchmark) triple with at least 80 items is one pair, giving 17 pairs, 3,800 labelled items, and success rates from 8.0% to 79.0% (Table 21 in the appendix). The subject dictionaries carry the real model names, providers, release dates and harness names, so the budget-0 prior sees realistic inputs; the raw records are released with the code.

Synthetic IRT testbed. Because the real testbed is small (small models, three benchmarks), we complement it with a generator that follows a two-parameter item response model [Lord, 1980, Embretson and Reise, 2000]. Subject s has ability θ_s correlated with its visible attributes (release year, reasoning effort, parameter count) plus noise; benchmark b has a location μ_b, a discrimination a_b and $K = 5$ topic clusters with difficulty offsets $c_k \sim \mathcal{N}(0, \sigma^2_{\text{topic}})$; item i in cluster $k(i)$ has difficulty $b_i = \mu_b + c_{k(i)} + \gamma\, z_{\text{len}}(i) + \varepsilon_i$ and $\Pr(y_{si} = 1) = \sigma(a_b(\theta_s - b_i))$. Item texts mix cluster-specific vocabulary (35% of tokens), benchmark template words, ambient vocabulary and numbers, so that lexical similarity carries difficulty information exactly when $\sigma_{\text{topic}} > 0$; half of the benchmarks expose a `tier=easy/medium/hard` feature derived from difficulty terciles. The main setting uses 10 subjects $\times$ 16 benchmarks $= 160$ pairs with 100–240 items each, $\sigma_{\text{topic}} = 0.70$ and $\gamma = 0.3$; a sweep varies both. Because the true probabilities are known, we can report two oracle references: the Brier score of the true item probabilities (the floor of any predictor) and that of the evaluation-set success rate used as a constant (the floor of any predictor without item-level information).

Experimental setup. Our rounds 1–3 harness reproduces the protocol of Section 2 with *per-pair* label lists (round 4 replaces this by the global list, Appendix D): items of each pair are split 50/50 with a seeded shuffle, the acquisition pool is streamed in a seeded random order, the policy under test selects up to 31 labels, and the nested prefixes of that one trajectory are passed to every predictor for every evaluation item, so all methods are compared on identical labels. The evaluator's default policy is reproduced as accept-with-probability labels_remaining/items_remaining. We report Brier ALC with 95% bootstrap confidence intervals over pairs (1,000 resamples), paired bootstrap differences between methods (same pairs, same trajectories), per-budget Brier scores and pooled ECE. Baselines: the constant 0.5; the organisers' empirical mean (their code: 0.5 without matching labels, the raw mean otherwise); our prior alone; shrinkage alone; shrinkage with each item-level component always on; the LOO-gated model (shipped in rounds 1–3).

G Rounds 1–3: results of the per-pair predictor

Main comparison. Table 15 and Figure 5 give the headline numbers. On the real testbed the per-pair predictor (shipped in rounds 1–3) reaches 0.2224 [0.1949, 0.2454] against 0.2871 [0.2369, 0.3303] for the empirical mean and 0.25 for the constant; the paired difference to the empirical mean is -0.0647 [-0.0872, -0.0415] (significant; the full model is better on 82% of pairs) and to the constant -0.0276 [-0.0506, -0.0050] (significant). On the synthetic testbed the corresponding ALCs are 0.1797 [0.1691, 0.1900] versus 0.2208 and 0.25, with paired differences -0.0411 [-0.0488, -0.0339] and

Table 15: Brier ALC (lower is better) with 95% bootstrap CIs over pairs and pooled ECE on the two local testbeds, all methods evaluated on identical random-policy acquisition trajectories. Oracles are references, not predictors.

	Synthetic IRT (160 pairs)		Real Ollama (17 pairs)	
Method	ALC [95% CI]	ECE	ALC [95% CI]	ECE
constant 0.5	0.2500 [0.2500, 0.2500]	0.111	0.2500 [0.2500, 0.2500]	0.127
empirical mean (organisers)	0.2208 [0.2055, 0.2362]	0.114	0.2871 [0.2369, 0.3303]	0.170
attribute prior only	0.2148 [0.2040, 0.2251]	0.041	0.2340 [0.2198, 0.2468]	0.076
prior + shrinkage	0.1797 [0.1691, 0.1900]	0.035	0.2228 [0.1951, 0.2460]	0.063
shrinkage + ridge (always on)	0.1814 [0.1708, 0.1918]	0.029	0.2199 [0.1945, 0.2416]	0.061
shrinkage + kNN (always on)	0.1794 [0.1687, 0.1895]	0.033	0.2226 [0.1949, 0.2453]	0.061
full: LOO-gated ridge + kNN (rounds 1–3)	0.1797 [0.1691, 0.1900]	0.032	0.2224 [0.1949, 0.2454]	0.061
oracle: pair success rate	0.1598 [0.1483, 0.1710]	0.000	0.1961 [0.1675, 0.2208]	0.000
oracle: true item probability	0.1379 [0.1280, 0.1482]	0.007	—	—

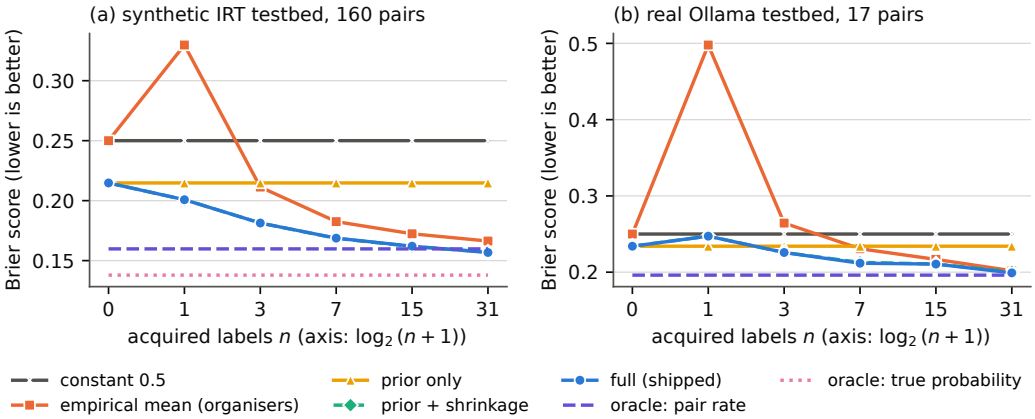


Figure 5: Brier score per budget (the ALC of Eq. 1 is the trapezoidal area under these curves on the $\log_2(n+1)$ axis). The empirical mean is worse than the constant at $n = 1$; shrinkage removes that penalty; the full model separates from shrinkage only at $n \geq 15$. Dashed/dotted lines are the oracle references.

-0.0703 [-0.0805, -0.0592] (both significant). The empirical mean is *worse* than the constant at $n = 1$ on both testbeds ($B_1 = 0.330$ and 0.498) and only overtakes it from $n = 3$ (synthetic) and $n = 7$ (real); on the real testbed its ALC (0.2871) is *above* the constant's 0.25 (paired difference +0.0371 [-0.0081, +0.0779], not significant).

Where the gain comes from. Decomposing the ablation rows of Table 15: the prior alone improves on the constant by -0.0352 (synthetic) and -0.0160 (real) ALC; shrinkage adds -0.0351 [-0.0415, -0.0285] and -0.0113 [-0.0283, +0.0042] on top (the real-testbed difference is not significant with 17 pairs). The item-level components are a much smaller effect. Against shrinkage, always-on kNN changes the ALC by -0.0003 (synthetic) and -0.0002 (real), always-on ridge by +0.0017 and -0.0029, and the LOO-gated full model by +0.0000 [-0.0002, +0.0003] (not significant) and -0.0004 [-0.0012, +0.0003] (not significant); at $n = 31$ alone the gated model changes the Brier score by -0.000 and -0.001. Replacing TF-IDF by `sentence-transformers/all-MiniLM-L6-v2` embeddings changes the full model's ALC by -0.0001 [-0.0005, +0.0001] (not significant), so we ship TF-IDF; Appendix H adds the LOO gate against always-on components and a sharper unigram TF-IDF, which makes the kNN worse. We report these as what they are: item-level adaptation from at most 31 labels is a marginal gain whose sign has to be protected by the gate.

Headroom. The oracle rows explain why. On the synthetic testbed, knowing the true probability of every item gives an ALC of 0.1379, knowing only the evaluation-set success rate 0.1598: perfect item-level discrimination is worth 0.0219 ALC, to be recovered from 15–31 labels at budgets that carry 30% of the weight. Shrinkage (0.1797) is 0.0199 above the rate oracle; that gap is the price of estimating a rate from few labels, concentrated at $n \leq 3$ (B_1: 0.041; B_{31}: -0.003), and only a

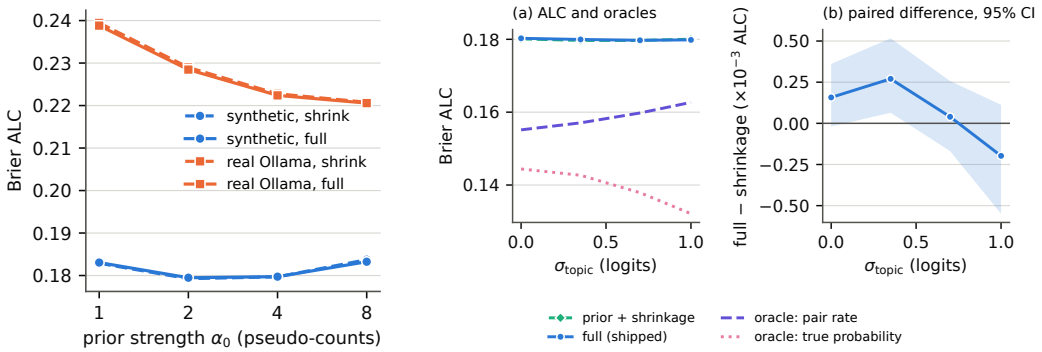


Figure 6: Left: Brier ALC vs. prior strength α_0 (dashed: shrinkage, solid: full model). Right: synthetic sweep of the topic-difficulty spread σ_{topic} at $\gamma = 0.3$: (a) ALC of shrinkage, the full model and the two oracles; (b) paired difference full − shrinkage with its 95% CI over pairs (negative favours the full model).

better prior, i.e. training data, can reduce it. On the real testbed the gap is 0.0266 (0.2228 vs. 0.1961). The two parts of the headroom are of similar size but call for different remedies: a prior fitted to real subjects and benchmarks, and item difficulty information from outside the pair.

Prior strength and text signal. Figure 6 gives the ALC for $\alpha_0 \in \{1, 2, 4, 8\}$ (shrinkage is best at 2 on synthetic and 8 on real; full-model range 0.0038 and 0.0182; we ship $\alpha_0 = 4$, the smallest worst-case regret and the best of $\{2, 4, 8\}$ on the training release) and the synthetic sweep over σ_{topic}, where the item-level effect stays two orders of magnitude below the item-level headroom (0.0080 to 0.0306): the gate protects against harm, it cannot create information.

Acquisition policies. None of the five policies we compared is significantly different from the evaluator's random policy on either testbed (Appendix H, Table 20): the novelty filter shipped in rounds 1–3 gives -0.0013 [-0.0038, +0.0013] (synthetic) and -0.0044 [-0.0108, +0.0025] (real), the organisers' uncertainty band +0.0001 and -0.0041. Selecting by $|p-0.5|$ distorts the pair-rate estimate that carries most of the metric; we ship the policy closest to random that removes near-duplicates.

Frontier-like subjects. The official test set has subjects our testbeds lack (2025–2026 releases, long reasoning efforts, very large models, agent harnesses); over 160 such descriptions the budget-0 prior ranges from 0.39 to 0.82 (the cap; Appendix L). What a wrong prior costs is exact under shrinkage (Table 18): a prior of 0.7 on a pair whose true rate is 0.3 costs 0.062 ALC and loses to the constant (excess 0.040), whereas shrinkage beats the constant whenever $|r-0.5| \geq 0.2$ and $|p_0-r| \leq 0.3$. On the real testbed an optimistic shift of every subject's prior logit costs significantly ($+0.5$: +0.0134), a pessimistic one does not (-0.5: -0.0034, not significant); on the training release the hand-set prior averages 0.510 against a mean pair rate of 0.537, so rounds 1–4 kept the intercept at zero (the selected submission changes it after the formative feedback, Section 4).

Replication and calibration. Appendix I reports replication over generator seeds and acquisition trajectories (the full model beats the empirical mean significantly in 4 of 4 further real trajectories). Pooled ECE (Table 15) is 0.032/0.061 (synthetic/real) for the full model against 0.114/0.170 for the empirical mean and 0.111/0.127 for the constant: shrinkage towards a sensible prior also fixes calibration at small budgets.

H Additional ablations

Prior strength. Table 24 gives the ALC for $\alpha_0 \in \{1, 2, 4, 8\}$. The best value differs between testbeds (shrinkage: 2 on synthetic, 8 on real), reflecting the wider spread of pair rates in the synthetic generator; the ALC of the full model varies by 0.0038 (synthetic) and 0.0182 (real) over the range, so the choice matters at the third decimal. On the training release $\alpha_0 = 4$ gives 0.2051 against 0.2052 (2) and 0.2086 (8).

When does text help? The synthetic sweep (Figure 6, right; Table 23) varies how much difficulty is explained by topic. With no topic or length signal ($\sigma_{\text{topic}} = 0.00$, $\gamma = 0.0$) the full model's paired difference to shrinkage is +0.0002 [-0.0000, +0.0005], the gate's cost when there is nothing to find; at the strongest setting ($\sigma_{\text{topic}} = 1.00$) it is -0.0002 [-0.0005, +0.0001], while the item-level headroom (rate oracle minus true-probability oracle) grows from 0.0080 to 0.0306.

Gating. Replacing the LOO gate by always-on components at their default settings gives 0.1808 versus 0.1797 (synthetic; paired gated−always -0.0011 [-0.0017, -0.0005], significant) and 0.2203 versus 0.2224 (real; +0.0021 [-0.0009, +0.0048], not significant). The gate pays where always-on components overfit (the synthetic ridge) and costs a little where a component carries real signal (on the real testbed longer GSM8K problems are failed more often, which the always-on ridge exploits at $n \geq 15$); the second effect is within noise. Requiring 0, 1 or 2 standard errors of LOO improvement gives full-model ALCs of 0.1808, 0.1799 and 0.1797 (synthetic) and 0.2219, 0.2223 and 0.2224 (real); we ship the most conservative setting.

Sentence embeddings. The embeddings were computed offline for the real testbed (the model is on the validator's allow-list via `models.txt`, so it could have been shipped). They change the full model's ALC from 0.2224 to 0.2223 (paired -0.0001 [-0.0005, +0.0001], not significant) and the always-on kNN variant from 0.2226 to 0.2252 (+0.0026 [+0.0000, +0.0055], significant), at the cost of a neural dependency, model loading and about 10 ms per embedding on CPU.

Sharper lexical similarity does not help. A unigram-only TF-IDF whose target terms are restricted to the acquired vocabulary separates same-topic from other-topic synthetic items far better (mean cosine about 0.3 versus 0.1 in the generator) but makes the always-on kNN *worse*: paired against the shipped representation, the always-on kNN changes by +0.0009 [-0.0001, +0.0020] (synthetic) and +0.0026 [+0.0006, +0.0050] (real), and the gated full model by +0.0004 [+0.0001, +0.0008] and +0.0006 [-0.0002, +0.0018]. With a handful of neighbours per topic the kernel-weighted residual is dominated by label noise; the bigram-diluted similarity acts as a regulariser. Item-level signal exists (the oracles show it) but 31 labels are too few to exploit it through similarity alone.

Acquisition policies. Table 20 compares policies for the full predictor on identical pools and streams. Plain stream order differs from the evaluator's random policy by -0.0013 [-0.0038, +0.0013] (synthetic, not significant) and -0.0044 [-0.0107, +0.0019] (real, not significant)—both are uniform samples, so this is the trajectory-to-trajectory noise floor of the comparison. The shipped novelty filter gives -0.0013 [-0.0038, +0.0013] and -0.0044 [-0.0108, +0.0025]. It does fire on the real testbed: on 10 of 17 pairs (40 candidates skipped in total, at most 10 per pair; templated BIG-Bench-Hard items such as *navigate* and *boolean expressions*, and SVAMP word problems sharing a template, reach cosine 0.98), and on none of the 160 synthetic pairs, whose pseudo-word items never get that similar—yet its ALC equals that of plain stream order to four decimals (0.2180 vs. 0.2180): skipping a near-duplicate changes which uniformly random items are labelled, not their distribution. The organisers' uncertainty band gives +0.0001 [-0.0027, +0.0030] and -0.0041 [-0.0096, +0.0016], and the novelty-plus-uncertainty variant -0.0013 and -0.0044: selecting by $|p - 0.5|$ reorders a near-random sample at small budgets and, once item-level corrections exist, biases it towards items of intermediate predicted difficulty, distorting the pair-rate estimate that carries most of the metric. None of the policies is significantly different from random on either testbed; we ship the one that is closest to random while removing redundant items.

I Replication

Regenerating the synthetic testbed with two further seeds gives full-model ALCs of 0.2266 and 0.1963 (empirical mean 0.2737 and 0.2438; shrinkage 0.2264 and 0.1963). On the real testbed, four further acquisition trajectories give full-model ALCs between 0.2127 and 0.2240 (empirical mean 0.2511–0.2841), and the full model beats the empirical mean significantly in 4 of 4 trajectories. A 3×3 grid over the kNN scale and anchor ($\kappa \in \{1, 2, 4\}$, $\tau \in \{1, 2, 4\}$) spans 0.1806–0.1809 on a smaller synthetic generator, i.e. the kernel constants matter little.

J Fitting pipeline details

`fit_prior_from_measurement_db.py` runs in five steps, each a sub-command. *plan*: list the repository, download the small `benchmarks` table of every directory and keep those with `response_type = binary` and `granularity = item` (the organisers' eligibility rule in `tools/prepare_data.py`); record the bytes still to download and stop above a cap (30 GB by default). *download*: fetch the eligible `subjects`, `items` and `responses` tables into the organisers' payload layout. *fit*: build one row per (subject, test condition, item) with the mean response over trials as label (fractional labels enter as two weighted rows), each pair receiving total weight one; compute the subject and item terms with the shipped `model.subject_terms` and `model.item_terms`, so that fitted coefficients map one-to-one onto the constants of Appendix E; add a one-hot for providers with at least 15 pairs and one for each training benchmark; fit an L_2-penalised logistic regression. Five-fold cross-validation holds out whole benchmarks; the penalty C and the logit cap are selected by the pair-weighted budget-0 Brier score of the prior on held-out pairs, and $\alpha_0 \in \{2, 4, 8\}$ by the closed-form shrinkage ALC of the protocol (50/50 split, 31 random acquisitions, Eq. 1) on held-out pairs with at least 80 items, paired against the hand-set prior on the same pairs. *testbeds*: evaluate the fitted table inside the harness on both local testbeds against the shipped constants. *dry-run*: generate tables with the release schema (shared pool of subjects, random subsets per benchmark, benchmark difficulty offsets with standard deviation one logit, labels from a known coefficient table over the prior's own terms, two ineligible benchmarks) and run *fit* on them. The dry run used 14 eligible benchmarks, 299 pairs and 47,784 rows; a light-regularisation fit recovers all 13 slope signs with maximum error 0.05 and mean error 0.02 logits, and the fitted benchmark effects have standard deviation 0.57. Its cross-validated selection chose $C = 0.003$ and $\alpha_0 = 4$ and did not beat the hand-set prior (budget-0 Brier +0.0036 [+0.0004, +0.0069]; shrinkage ALC +0.0013 [+0.0001, +0.0027]), so on these tables the rule would have kept the hand-set constants.

The run on the release. Revision `54074453da`; 163 eligible directories, 144 with at least one pair of 20 or more distinct items; the pair unit is (benchmark directory, subject, test condition), since the release records several test conditions per subject on many benchmarks; 16,718 pairs, 3,118,742 rows (each pair capped at 200 items); 22 providers with at least 15 pairs received a coefficient. Pair success rates have mean 0.537 and standard deviation 0.265 across pairs (benchmark means: standard deviation 0.223), while the hand-set p_0 has mean 0.510 and standard deviation 0.072. The penalty grid ran from $C = 0.001$ to 1; the selection took $C = 0.001$, cap 1.5 and $\alpha_0 = 2$ (cross-validated budget-0 Brier 0.2421, shrinkage ALC 0.2061). The fitted benchmark effects have standard deviation 0.46 logits. Fitted vs. effective hand-set coefficients: release year +0.14 vs. +0.24 per year, size +0.08 vs. +0.17 per doubling, hard-tier cue -0.65 vs. -0.60, easy-tier cue +0.08 vs. +0.50, item length +0.08 vs. +0.12; the reasoning-effort and harness terms shrink to about zero (-0.08, -0.02; 58% of pairs carry no provider field and most carry no effort or harness field), and the provider table spans -0.23 (`hugging face`) to +0.55 (`unknown`). Inside the harness on the local testbeds the fitted table changes the full model's ALC by +0.0021 [-0.0006, +0.0045] (real) and +0.0058 [+0.0032, +0.0081] (synthetic); the result files are `prior_fit.json` and `prior_fit_testbeds.json`, and the ship rule's verdict is recorded there (not shipped).

K Item-text difficulty prior: details

Pre-registration and outcome. `experiments/results/round3/PREREG.md` (SHA-256 prefix `bb54af425d`, recorded in every result file and verified unchanged at build time) fixed features, target, model, nested selection, evaluation and ship rule before any text model was fitted. `PREREG_ADDENDUM_1.md` (verified unchanged) registered the two exploratory variants after the primary folds had run, explicitly without influence on the decision. The rule: ship iff (a) the benchmark-held-out ALC difference (text minus hand-set) has a 95% cluster-bootstrap CI below zero, (b) the real testbed is not significantly worse, and the engineering gates hold (≥ 200 calls/s on one core with a new target item per call, weights ≤ 5 MB, validator and smoke test pass, never raises). Outcome: (a) fails, (b) holds, gates pass; the text prior is not shipped.

Features and target. JSON contents (the release stores many items as JSON objects) are flattened into `key: value` lines; words and word bigrams come from the first 2,000 characters (digit runs mapped to a length token), character trigrams from the first 500, and `item_features` tokens form a

fourth family; each family is sublinear-tf weighted and ℓ_2-normalised, then hashed with `zlib.crc32` (sign from bit 31) into 262,144 buckets. The target of item i is $\text{logit}\,\frac{n_i r_i + 0.5}{n_i + 1}$ minus the mean hand-set subject logit of the n_i pairs that evaluated it and minus its hand-set item logit, clipped to ± 5. At most 1,000 items per benchmark (109,233 in total) are used for fitting; all 515,067 items of the frame are scored (237,628,268 non-zeros). Exact duplicates of a test item's content in a training benchmark are rare (0.02% of frame items on average), so shared items do not inflate the cross-validation.

Nested selection and fits. In each outer fold a four-fold benchmark-grouped inner CV chooses the ridge penalty from $\{0.03, 0.1, 0.3, 1, 3, 10\}$ by weighted MSE of the target; the inner out-of-fold scores give W, B (pair-weighted logistic likelihood), the cap ($\{1.5, 2.5, 3.5\}$, training budget-0 Brier) and the within-pair weight λ of the text offset ($\{0, 0.5, 1\}$, training closed-form ALC). Selected: penalty 10 in every fold (the grid maximum), caps 1.5, 2.5, $\lambda = 0$, W from -0.32 to -0.03 and B from +0.06 to +0.21. The held-out R^2 of the residual target was -0.06 to 0.04 across folds (inner CV: -0.04 to -0.00). The final fit on all 144 benchmarks again selected penalty 10 with $W = -0.20$, $B = +0.14$, cap 1.5 and $\lambda = 0$; it is kept as `text_prior_candidate.json` (1.40 MB) and not shipped.

Table 16: Item-text prior: benchmark-held-out cross-validation on the training release (closed-form protocol, shrinkage with $\alpha_0 = 4$ unless stated, no item-level components). B_0 over all items of each of the 16,718 pairs; B_1–B_7 and ALC on the evaluation half of the 16,303 pairs with at least 80 items. Exploratory rows were registered after the primary result; the oracles are references.

Method	B_0	B_1	B_3	B_7	ALC
hand-set prior (shipped)	0.2388	0.2235	0.2083	0.1956	0.2051
hand-set + item-text prior (pre-registered)	0.2391	0.2234	0.2083	0.1956	0.2051
same, $\alpha_0 = 2$	0.2391	0.2240	0.2086	0.1955	0.2052
same, $\alpha_0 = 8$	0.2391	0.2278	0.2141	0.2002	0.2086
exploratory E1: penalty up to 10^4	0.2397	0.2239	0.2085	0.1957	0.2053
exploratory E2: 16 statistics only	0.2395	0.2237	0.2084	0.1956	0.2052
hand-set + fitted intercept shift only	0.2398	0.2240	0.2086	0.1957	0.2054
intercept only (training mean)	0.2519	0.2320	0.2127	0.1975	0.2096
oracle: held-out benchmark mean	0.2187	0.2115	0.2022	0.1932	0.1990
oracle: pair success rate	–	0.1756	0.1756	0.1756	–

Results. Table 16. Per budget the paired differences are -0.0000 [-0.0025, +0.0027] at $n = 1$ and -0.0000 [-0.0013, +0.0014] at $n = 3$ (gap to the pair-rate oracle closed: 0.1% [-5.6, 6.0] and 0.1% [-4.2, 4.3]); budget-0 Brier changes by +0.0003 [-0.0036, +0.0046]; the plain pair bootstrap gives -0.0000 [-0.0001, +0.0001]. The text prior has the lower mean ALC in 1 of 5 folds (significantly in 0) and on 57 of 135 benchmarks. Refitting only the intercept shift ($W = 0$; a descriptive decomposition) gives +0.0003 [-0.0007, +0.0009]: the hand-set prior's level is already right on held-out benchmarks. With the text prior, $\alpha_0 = 2$ gives +0.0001 and $\alpha_0 = 8$ +0.0035 (significant) relative to the hand-set prior at $\alpha_0 = 4$. The exploratory variants (E1: grid 10, 100, 1000, 10000, selected 100, 1000, 10000; E2: statistics only, grid 0.1, 1, 10, 100, selected 100) changed the ALC by +0.0002 [-0.0007, +0.0011] and +0.0001 [-0.0010, +0.0014] and again had negative W in 5 and 5 of 5 folds. The held-out benchmark-mean oracle closes 30%, 25% and 19% of the gap at $n = 0, 1, 3$.

Local testbeds. Full harness, identical trajectories: real ALC 0.2224 (hand-set) vs. 0.2243 (text), paired +0.0019 [-0.0002, +0.0037], text better on 35% of pairs; without the BIG-Bench-Hard pairs, whose items may overlap the `fewshot_ttt_bbh` training benchmark, +0.0017 [-0.0007, +0.0042] (13 pairs). Synthetic: 0.1797 vs. 0.1800 (+0.0003 [-0.0002, +0.0008]); its pseudo-word items carry no transferable text signal by construction.

Engineering gates and the shipped file. With the candidate loaded, `predict` runs at 3,056, 673 and 417 calls/s on one core with a new 15-, 300- and 2,000-word target item per call at budget 31 (the per-item text score is cached), the weights load in 22 ms, and the smoke test passes 44 cases, including malformed JSON contents and a corrupted weight specification (which falls back to the hand-set prior); standard-library and numpy scores agree to 9.0×10^{-9}. Since the rule failed, the round-3 submission ZIP was unchanged (SHA-256 prefix `3f0d15c879d9`); the development `model.py` kept the text-prior hooks, which are inactive without a weight file and reproduced the then-shipped file's predictions bit for bit on 5,544 of 5,544 test cases.

Benchmarks of the training release used (144). abstract_reason, adaptivestep, advprompter, aegis, afrieval, afrimedqa, agentic_review_perturbation, agi_elo, ai2d_test, aider, algotune, alignment_faking, alpha_sql, appworld, auditing_sabotage_bench, averimatec, babilong, bbq, beavertails, benger, bertaqa, bird_sql, brace, braveguard, bridging_gap, causal_axioms, cellverse, ceval, chi_bench, clasheval, cmmlu, critic_discernment_game, crow, cruxeval, dave, devbench, editbench, edu_circuit_hw, egoschema, ehrflowbench, elicitation_game, emoji_attack, engdesign, enginemt_qa, erbench, evouna, exploitgym, faithcot, felm, fewshot_ttt_bbh, financebench, finegrain_t2i, haiid, hallusionbench, harmbench, healthadminbench, helm_afr, helm_bbq, helm_thaiexam, hle, icpc2_code_selector, igakuqa119, ikp, interchangeable_token_embeddings, jailbreakbench, judgetuning, kernelbench, kmmlu, kormedmcqa, legal_rag_bench, legalbench, lingoly, liveaopsbench, llm_bp_tag, llm_uncertainty_bench, llmail_inject, llms_lean_formalization, mathconstraint, mathvista_mini, medhelm_ehr_sql, medhelm_head_qa, medhelm_med_mcqa, medhelm_med_qa, medhelm_medbullets, medhelm_medcalc_bench, medhelm_medec, medhelm_medhallu, medhelm_pubmed_qa, medhelm_race_based_med, mind2web, mj_bench, mmbench_v11, mmlu, mmlupro, monkey_power_laws, morphkv, multi_swebench, mypcbench, naep_llm_students, naturebench, nester, nyu_ctf_bench, ood_prediction, openbiorq, os_harm, perfcodebench, perumedqa, planbench, proofaug, prox, pxplore, real_webagents, realtimeqa, reliancescope, researchcodebench, revolve, risebench, robust_reasoning_benchmark, safeagentbench, sgrades, sib200, sketchjudge, software_world_models, statqa, subjective_qa, sugarcrepe, sustainable_food, swe_rebench, swe_smith, swe_together, swebench_live, swepolybench, tau2_bench, tensortrust, truthfulqa_mc, visual_memory, visual_riddles, visualwebarena, vl_rewardbench, weavebench, wikihow_agent, workarena, workbench_revisited, worldcentralbanks.

L Prior sensitivity for frontier-like subjects

Table 17 gives the budget-0 prior p_0 of the round-4 constants (1010370; in the selected submission p_0 no longer depends on the subject) for subject descriptions that the official test set is likely to contain and our testbeds do not. Table 18 gives the exact expected cost of a miscalibrated prior under the per-pair shrinkage rule, and Table 19 the measured effect of shifting every subject's prior logit on the two testbeds (`PRIOR_INTERCEPT` in `model.py`; all other constants unchanged; identical trajectories; generated by `prior_sensitivity.py`).

Discussion. The official test set has subjects our testbeds lack (2025–2026 releases, long reasoning efforts, very large models, agent harnesses). Appendix L tabulates the budget-0 prior over 160 such descriptions: p_0 ranges from 0.39 to 0.82 (the cap); the organisers' GPT-4o and 2026 medium-effort sample subjects receive 0.49 and 0.66, our testbed models 0.36–0.39. What a wrong prior costs is exact: with $\alpha_0 = 4$ and n labels the expected Brier excess over the pair-rate oracle is $\mathbb{E}[(\rho_n - r)^2]$ with binomial successes (Table 18). A prior of 0.7 on a pair whose true rate is 0.3 costs 0.062 ALC and loses to the constant (excess 0.040); shrinkage beats the constant whenever $|r - 0.5| \geq 0.2$ and $|p_0 - r| \leq 0.3$, and never when $|r - 0.5| < 0.15$, where the constant is close to optimal. Shifting every subject's prior logit by ± 0.5 or ± 1 (Table 19) is asymmetric on the real testbed: $+0.5$ and $+1$ cost +0.0134 and +0.0353 ALC (both significant), whereas -0.5 changes it by -0.0034 [-0.0117, +0.0049] (not significant)—small models at 8.0–79.0% accuracy tolerate a pessimistic prior; on the synthetic testbed zero is best and both ± 0.5 shifts cost significantly (+0.0037, +0.0047). On the training release the hand-set prior averages 0.510 against a mean pair rate of 0.537, i.e. it is not optimistic for real systems *on average over the release*. Rounds 1–4 kept the intercept at zero and named it as the first constant to change if the official formative score pointed the other way. It did: on the hidden pool the budget-0 prior was worse than the constant, and the selected submission lowers the intercept and switches the subject terms off (Section 4).

M Validator and smoke-test output

For the selected ZIP (1019682) the organisers' validator passes (full output in `experiments/results/w2_items/candidates/w2_f05s1_k1q2/validator_output.txt`). For the base ZIP (1010717) it passes (exit code 0, "`OK: ...xp_level_m050.zip looks like a valid submission ZIP.`"; full output in `experiments/results/deep_dive/candidates/xp_level_m050/validator_output.txt`). For the round-4 ZIP (1010370) it prints

Table 17: Budget-0 prior p_0 (no item cues; 600-character item) by release year and `reasoning_effort` for a subject with no size or name cue on the default harness (left), and for a high-effort subject on a non-default harness with a *mini* name cue, a 70B or a 400B parameter count (right). The organisers' sample subjects receive 0.49 (GPT-4o, 2024-05) and 0.66 (2026-01, medium effort, non-default harness).

	default harness, no name cue				high effort, non-default harness		
Release year	none	low	medium	high	mini	70B	400B
2023	0.44	0.46	0.49	0.51	0.48	0.58	0.69
2024	0.50	0.52	0.55	0.58	0.54	0.64	0.74
2025	0.56	0.58	0.61	0.63	0.60	0.70	0.78
2026	0.62	0.64	0.67	0.69	0.66	0.75	0.82

Table 18: Exact expected Brier ALC excess over the pair-rate oracle $r(1-r)$ of the per-pair shrinkage rule ($\alpha_0 = 4$, clipping to $[0.01, 0.99]$) when the prior rate is p_0 and the true pair rate is r; the number of successes among the n acquired labels is binomial(n, r) and the ALC weights are those of Eq. 1. The last column is the excess of the constant 0.5; † marks cells where the constant is better.

	prior rate p_0						
True rate r	0.3	0.4	0.5	0.6	0.7	0.8	constant 0.5
0.2	0.010	0.020	0.036	0.060	0.089	0.126†	0.090
0.3	0.009	0.012	0.022	0.039	0.062†	0.091†	0.040
0.4	0.013†	0.010†	0.013†	0.023†	0.040†	0.063†	0.010
0.5	0.024†	0.014†	0.011†	0.014†	0.024†	0.040†	0.000
0.6	0.040†	0.023†	0.013†	0.010†	0.013†	0.023†	0.010
0.7	0.062†	0.039	0.022	0.012	0.009	0.012	0.040
0.8	0.089	0.060	0.036	0.020	0.010	0.007	0.090

```
OK: paiec_submission.zip looks like a valid submission ZIP.
```

(full output in `experiments/results/validator_output.txt`; ZIP SHA-256 prefix `c76331d084f1`). The smoke test (`experiments/results/smoke_test.json`) records 44 successful cases and the timings of the per-pair path quoted in Appendix E. One case times a 62,000-label checkpoint and fails if the first call takes more than 60 s or later calls more than 5 ms each; on a slower or battery-powered machine this timing check can fail while every functional case passes.

N Reproduction commands

Commands run from `experiments/src` with Python 3: the standard library for the submission; numpy, scipy, pandas and pyarrow for the simulators; `huggingface_hub` and scikit-learn for the download and fit. They use `-I` because the simulator reads downloaded data.

```
# 1. Training release (Hugging Face login, dataset terms accepted); round-4 frame and simulator.
python -I fit_prior_from_measurement_db.py plan --payload DIR
python -I fit_prior_from_measurement_db.py download --payload DIR
python -I xpair_sim.py build --payload DIR   # then: cv, addendum, acq, realtest, runtime, equivalence
# 2. Base ZIP from submission/paiec_submission.zip (three constant lines), with the organisers' validator
#    (commit cc88a959), our smoke test and official_loop_check.py (organisers' run_streaming client).
```

Table 19: Brier ALC of the prior alone, shrinkage and the full model when every subject's prior logit is shifted by `PRIOR_INTERCEPT` (random policy, same trajectories as Table 15). Round-4 value: 0; the selected submission uses -0.5 with the subject terms switched off, which this table does not cover.

	Real Ollama (17 pairs)			Synthetic IRT (160 pairs)		
Shift (logits)	prior	shrinkage	full	prior	shrinkage	full
-1.0	0.2511	0.2229	0.2229	0.2689	0.1959	0.1955
-0.5	0.2319	0.2191	0.2190	0.2286	0.1835	0.1834
$+0.0$	0.2340	0.2228	0.2224	0.2148	0.1797	0.1797
$+0.5$	0.2663	0.2364	0.2358	0.2268	0.1844	0.1844
$+1.0$	0.3250	0.2585	0.2576	0.2527	0.1932	0.1932

```
python -I deep_dive/build_candidate.py xp_level_m050 --intercept -0.5 --subject-shrink 0 --tau-mu 2
python -I deep_dive/proto_level3.py          # Table 1; also ece_check, proto_oracle, w1 scripts
# 3. Item layer: extract items, tune (folds 0-1), select, confirm (folds 2-4), build and check the ZIP.
python -I w2_items/extract_all.py
python -I w2_items/run_w2.py tune F;  python -I w2_items/run_w2.py tune K
python -I w2_items/run_w2.py select;  python -I w2_items/run_w2.py confirm
python -I w2_items/build_w2.py w2_f05s1_k1q2 --tau-feat 0.5 --share 1 --tau-kern 1 --q 2 --checks
python -I w2_items/w2_equivalence.py w2_f05s1_k1q2
#    Codabench 17828 starting kit, in its directory (we ran its validator, not its smoke test):
python tools/check_submission_zip.py w2_f05s1_k1q2.zip;  python tools/run_smoke_test.py w2_f05s1_k1q2/
# 4. Formative aggregates from saved Codabench scoring logs (private to the submitter); the report.
python -I deep_dive/formative_summary.py;    python -I w2_items/w2_draws.py
bash build_paper.sh                          # every number, table, figure and the claim audit
```

`plan` lists the *current* head of the dataset; our results used revision `54074453da`. `DIR` is `../_downloads/measurement_db_payload` next to the repository. The INV populations also need the organisers' benchmark inventory as a CSV file, named by `PAIEC_INV_CSV`; all other paths are relative to the repository. Seeds are fixed; we used Python 3.14.5. Per-unit simulator outputs (`*.npz`) are not in the repository, so CIs are recomputed by rerunning step 3. Step 4 needs our private Codabench logs. Without them, the audit's log-based checks (draw aggregates, retry timing) fail, but the released aggregates (`draws.json` and cached summaries) still produce every number of the report. Hardware: one laptop CPU (8 cores, 34 GB RAM, no GPU; the frame build needs about 12 GB free); the item ratings ran on a Mac GPU (MPS).

O Additional tables

Table 20: Acquisition policies (full predictor): Brier ALC [95% CI] and paired difference to the evaluator's random policy on the same pairs (negative favours the policy). The BLE rule accepts a candidate when its current prediction satisfies $|p - 0.5| \leq 0.15$ [Truong et al., 2026].

	Synthetic		Real	
Policy	ALC [95% CI]	Δ vs random [95% CI]	ALC [95% CI]	Δ vs random [95% CI]
random (evaluator default)	0.1797 [0.1691, 0.1900]	—	0.2224 [0.1949, 0.2454]	—
stream order	0.1784 [0.1675, 0.1888]	-0.0013 [-0.0038, 0.0013]	0.2180 [0.1915, 0.2400]	-0.0044 [-0.0107, 0.0019]
novelty filter (rounds 1–3)	0.1784 [0.1675, 0.1888]	-0.0013 [-0.0038, 0.0013]	0.2180 [0.1912, 0.2407]	-0.0044 [-0.0108, 0.0025]
novelty + uncertainty	0.1785 [0.1675, 0.1888]	-0.0013 [-0.0038, 0.0013]	0.2180 [0.1912, 0.2407]	-0.0044 [-0.0108, 0.0025]
uncertainty band (BLE rule)	0.1798 [0.1693, 0.1896]	+0.0001 [-0.0027, 0.0030]	0.2183 [0.1905, 0.2409]	-0.0041 [-0.0096, 0.0016]

Table 21: Real Ollama testbed: one pair per (model, harness, benchmark). Accuracy is the success rate over all items of the pair (acquisition and evaluation pools together).

Model (Ollama tag)	Harness	Benchmark	Items	Acc. (%)
qwen2.5:1.5b-instruct	8-shot CoT	GSM8K (test, first 200)	200	37.5
qwen2.5:3b-instruct	8-shot CoT	GSM8K (test, first 200)	200	24.0
qwen2.5:7b-instruct	8-shot CoT	GSM8K (test, first 200)	200	42.5
qwen2.5:1.5b-instruct	8-shot CoT	SVAMP (test, first 200)	200	79.0
qwen2.5:3b-instruct	8-shot CoT	SVAMP (test, first 200)	200	67.0
llama3.2:3b	direct answer	BBH mix (10 subtasks × 30)	300	49.3
qwen2.5:1.5b-instruct	direct answer	BBH mix (10 subtasks × 30)	300	39.3
qwen2.5:3b-instruct	direct answer	BBH mix (10 subtasks × 30)	300	29.3
qwen2.5:7b-instruct	direct answer	BBH mix (10 subtasks × 30)	300	51.0
llama3.2:3b	direct answer	GSM8K (test, first 200)	200	8.0
qwen2.5:1.5b-instruct	direct answer	GSM8K (test, first 200)	200	12.0
qwen2.5:3b-instruct	direct answer	GSM8K (test, first 200)	200	12.5
qwen2.5:7b-instruct	direct answer	GSM8K (test, first 200)	200	23.5
llama3.2:3b	direct answer	SVAMP (test, first 200)	200	33.0
qwen2.5:1.5b-instruct	direct answer	SVAMP (test, first 200)	200	32.5
qwen2.5:3b-instruct	direct answer	SVAMP (test, first 200)	200	39.5
qwen2.5:7b-instruct	direct answer	SVAMP (test, first 200)	200	64.5

Table 22: Brier score per budget for the main methods (random policy). The last column is the ALC of Eq. 1.

Testbed	Method	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC
synthetic	constant 0.5	0.250	0.250	0.250	0.250	0.250	0.250	0.2500
	empirical mean	0.250	0.330	0.211	0.183	0.172	0.166	0.2208
	attribute prior only	0.215	0.215	0.215	0.215	0.215	0.215	0.2148
	prior + shrinkage	0.215	0.201	0.181	0.169	0.162	0.157	0.1797
	full (shipped)	0.215	0.201	0.181	0.169	0.162	0.157	0.1797
	oracle: pair success rate	0.160	0.160	0.160	0.160	0.160	0.160	0.1598
real	constant 0.5	0.250	0.250	0.250	0.250	0.250	0.250	0.2500
	empirical mean	0.250	0.498	0.264	0.231	0.217	0.202	0.2871
	attribute prior only	0.234	0.234	0.234	0.234	0.234	0.234	0.2340
	prior + shrinkage	0.234	0.247	0.226	0.213	0.211	0.201	0.2228
	full (shipped)	0.234	0.247	0.226	0.212	0.211	0.199	0.2224
	oracle: pair success rate	0.196	0.196	0.196	0.196	0.196	0.196	0.1961

Table 23: Synthetic signal sweep (160 pairs per setting, seed 0): ALC of shrinkage and of the full model, their paired difference, and the two oracle references.

σ_{topic}	γ	Shrinkage	Full [95% CI]	Full − shrink [95% CI]	Oracle rate	Oracle p
0.00	0.0	0.1816	0.1818 [0.1708, 0.1926]	+0.0002 [-0.0000, 0.0005]	0.1543	0.1464
0.00	0.3	0.1801	0.1803 [0.1692, 0.1906]	+0.0002 [-0.0000, 0.0004]	0.1551	0.1444
0.35	0.3	0.1797	0.1800 [0.1689, 0.1905]	+0.0003 [0.0001, 0.0005]	0.1571	0.1427
0.70	0.3	0.1797	0.1797 [0.1691, 0.1900]	+0.0000 [-0.0002, 0.0003]	0.1598	0.1379
1.00	0.3	0.1800	0.1798 [0.1694, 0.1898]	-0.0002 [-0.0005, 0.0001]	0.1627	0.1321
0.70	0.0	0.1793	0.1796 [0.1688, 0.1896]	+0.0002 [0.0000, 0.0005]	0.1583	0.1392

Table 24: Sensitivity to the prior strength α_0 (Brier ALC).

	$\alpha_0 = 1$	$\alpha_0 = 2$	$\alpha_0 = 4$	$\alpha_0 = 8$
Synthetic, shrinkage	0.1829	0.1793	0.1797	0.1837
Synthetic, full	0.1831	0.1795	0.1797	0.1832
Real, shrinkage	0.2394	0.2290	0.2228	0.2208
Real, full	0.2388	0.2285	0.2224	0.2206